



Universidad
Internacional
de Valencia

Desarrollo de modelos para la imitación del comportamiento humano en el juego de Ajedrez

Titulación:
Máster Universitario en
Inteligencia Artificial

Curso académico
2023 – 2024

Alumno/a: Gordillo Sánchez,
Pablo

D.N.I: 77199955H

Director/a de TFM: Yaneth
Coromoto Moreno Caldera

Convocatoria: Tercera

30 de septiembre de 2024

De:
 Planeta Formación y Universidades

*Nothing is so painful to the human mind as a great
and sudden change.*

Mary Shelley

Índice general

Índice de figuras	III
Índice de tablas	v
Resumen	1
1. Introducción	3
1.1. Planteamiento del problema	3
1.2. Justificación	4
1.3. Objetivos	5
1.4. Alcance	6
1.5. Estructura	6
2. Estado del Arte	7
2.1. Motores tradicionales	7
2.2. Motores basados en redes neuronales	8
2.3. Human-like engines	9
2.4. Herramientas de juego de ajedrez	11
3. Marco teórico	13
3.1. Machine Learning y paradigmas de aprendizaje	13
3.2. Deep Learning	14
3.2.1. MLP	14
3.2.2. CNN	15
3.2.3. Conceptos fundamentales del proceso de entrenamiento	16
3.3. Arquitectura del human-like engine	18
3.4. Notación y comunicación de jugadas de ajedrez	19
3.4.1. Notación Algebraica	19
3.4.2. Notación FEN	20
3.4.3. ELO	22
4. Metodología y materiales	23

4.1. Metodología	23
4.2. Materiales	25
4.2.1. Datos	26
4.2.2. Herramientas y <i>hardware</i>	27
5. Desarrollo y resultados	29
5.1. Estudio y preparación de los datos	29
5.1.1. Entendimiento y limpieza de los datos	29
5.1.2. Extracción de características	34
5.1.3. Gestión de los datos para el entrenamiento	38
5.2. Desarrollo de los modelos	39
5.2.1. Modelo de predicción de pieza a mover	39
5.2.2. Modelos de predicción de posición final	43
5.2.3. Modelos de predicción de movimiento completo	47
5.2.4. Modelo de predicción de error	52
5.3. Evaluación y comparación de resultados	53
5.3.1. Evaluación del modelo de predicción de movimientos	54
5.3.2. Evaluación del modelo de predicción de error	57
5.4. Despliegue del modelo	58
5.4.1. Integración con Python-Chess	58
5.4.2. Despliegue en Lichess	59
6. Conclusiones, limitaciones y perspectivas de futuro	65
6.1. Conclusiones	65
6.2. Limitaciones	66
6.3. Líneas futuras	67
A. Apéndice A	69
Bibliografía	70

Índice de figuras

2.1. Garry Kasparov contra Deep Blue (1996). Fuente: Worrall (2018)	7
2.2. <i>Accuracy</i> obtenida por las versiones de Maia según ELO. Fuente: McIlroy-Young et al. (2020)	10
2.3. <i>Accuracy</i> obtenida para distintas versiones de Maia. Transfer Maia corresponde al modelo personalizado para un jugador. Fuente: McIlroy-Young et al. (2022)	10
2.4. Interfaz Gráfica de Arena Chess	11
3.1. Representación de una neurona en un MLP. Fuente: Amanatullah (2023)	15
3.2. Funciones de activación más comunes. Fuente: Loaiza (2020)	15
3.3. Ejemplo de kernel para obtener bordes de una imagen. Fuente: Kumar (2020)	16
3.4. Curvas de pérdida <i>underfitted</i>	17
3.5. Curvas de pérdida <i>overfitted</i>	17
3.6. Arquitectura de red Maia para predicción del movimiento. Fuente: McIlroy-Young et al. (2020)	18
3.7. Posición inicial	19
3.8. Casilla b3	20
3.9. Nf3	20
3.10. Qf7++	20
3.11. Posición de jaque mate para blancas	21
3.12. Distribución de jugadores por ELO en <i>Rapid</i> en Lichess.com. Fuente: Lichess (2024)	22
4.1. Diagrama de metodología CRISP-DM de Wirth y Hipp (2000)	23
5.1. Agrupación de entradas por nivel para una muestra de 1 millón.	30
5.2. Distribución de frecuencia de movimientos	32
5.3. Movimientos ordenados por frecuencia	33
5.4. Frecuencia de errores según evaluación de <i>Stockfish</i>	33
5.5. Frecuencia de casilla inicial.	36
5.6. Frecuencia de casilla final.	37
5.7. Arquitectura de Modelo predicción posición inicial.	40

5.8. Pérdidas de entrenamiento de modelo inicial	41
5.9. Accuracy de entrenamiento de modelo inicial	42
5.10. Pérdidas de entrenamiento de modelo sin <i>Dropout</i>	43
5.11. Accuracy de entrenamiento de modelo sin <i>Dropout</i>	43
5.12. Arquitectura de Modelo predicción de posiciones inicial y final.	44
5.13. Proceso de entrenamiento de predicciones inicial y final	45
5.14. Distribución de predicciones de posición final utilizando el segundo modelo. . .	46
5.15. Pérdidas de entrenamiento de modelo 4	49
5.16. Accuracy de entrenamiento de modelo 4	49
5.17. Pérdidas de entrenamiento de modelo 4 mejorado	50
5.18. Accuracy de entrenamiento de modelo 4 mejorado	51
5.19. Arquitectura de Modelo predicción de movimiento completo.	51
5.20. Proceso de entrenamiento de modelo final.	52
5.21. Pérdidas de entrenamiento de modelo de predicción de error	53
5.22. Accuracy de entrenamiento de modelo de predicción de error	53
5.23. Proceso de entrenamiento de para Blancas y Negras.	54
5.24. Curvas PR y ROC para la predicción de error	58
5.25. Ejemplo de primeros 5 movimientos en una partida.	60
5.26. Partida entre nuestro motor (Negras) y Maia 1900 (Blancas) terminada en tablas.	64

Índice de tablas

5.1. Tabla “moves” que recoge las entradas del <i>dataset</i>	31
5.2. Tabla “moves” que recoge las entradas del <i>dataset</i> transformadas.	38
5.3. Comparación de arquitecturas entrenadas con un subconjunto de los datos . .	48
5.4. Evaluación del modelo por clases.	55
5.5. Evaluación de movimientos legales	55
5.6. Accuracy de los modelos del estado del arte.	56
5.7. Matriz de confusión. Se expresa el porcentaje con respecto al total de instancias esperadas de esa clase.	57
5.8. Comparación de distintos umbrales	57



Resumen

La popularidad del ajedrez ha crecido mucho en los últimos años, especialmente influenciado por su aparición en medios audiovisuales populares. La creación de comunidades *online* de Ajedrez alrededor de personalidades de Internet y Grandes Maestros del juego, han propiciado que un gran número de jugadores se hayan acercado por primera vez al mismo. Estos jugadores sin conocimientos previos en el juego y sin una figura de profesor tradicional han tenido algunas dificultades para aprender y mejorar.

El propósito de este Trabajo Fin de Máster es estudiar y desarrollar modelos de inteligencia artificial que puedan ser utilizados en motores de ajedrez y sean útiles para jugadores noveles. A diferencia de con los motores tradicionales, no se busca implementar motores que realicen siempre la mejor jugada, ya que estos movimientos son difícilmente comprensibles sin un conocimiento profundo. Se encuentra en los motores *human-like* una posible solución a este problema, ya que intentan imitar el comportamiento humano en sus jugadas.

En concreto se ha implementado, aplicando metodología CRISP-DM, un modelo de *Deep Learning* que implementa una red neuronal convolucional capaz de predecir el movimiento jugado por un humano mediante aprendizaje supervisado. Para acotar el problema, este modelo es entrenado con partidas del portal web Lichess de jugadores de 1700 puntos ELO, aunque su arquitectura puede ser usada para jugadores de cualquier nivel. Así, también se busca predecir errores comunes de jugadores de este nivel, para lo cual se implementa un modelo auxiliar que se especialice en esta tarea.

Finalmente, se consigue una tasa de aciertos en la predicción de la jugada de un 44,41 %, comparable a resultados de otros motores del estado del arte y utilizando una cantidad menor de datos. Con estos resultados, se ha creado un *bot* en el portal web Lichess del que se han extraído los datos para que cualquier usuario pueda jugar contra él, llamado BlackBishop1700. La implementación de este *bot* ha requerido de hacer uso de la API de la web y realizar ciertos ajustes al motor de ajedrez para brindar una experiencia satisfactoria al usuario. La página web permite realizar evaluaciones posteriores de los movimientos jugados por el jugador y la máquina, por lo que se complementa el propósito de este proyecto. Podemos concluir que la elaboración de motores *human-like* todavía tienen muchas oportunidades de mejora y formas distintas de aplicación e integración en otras herramientas.

Abstract

Chess popularity has significantly increased in recent years, especially due to its presence in popular audiovisual media. The creation of online chess communities around Internet personalities and Chess Grandmasters has led a large number of players to approach chess for the first time. These players, lacking prior knowledge of the game and without the guidance of a traditional coach, have encountered some difficulties in learning and improving.

The aim of this Master's Thesis is to study and develop artificial intelligence models that can be used in chess engines and be helpful for novice players. Unlike traditional engines, the goal is not to implement engines that always make the best move, as these moves are often hard to understand without deep knowledge. Human-like engines present themselves as a possible solution to this problem, attempting to mimic human behavior in their moves.

Specifically, a deep learning model has been implemented using the CRISP-DM methodology, which features a convolutional neural network capable of predicting moves made by a human through supervised learning. To narrow the scope of the problem, this model is trained using games from the Lichess website played by users with 1700 ELO points, although its architecture can be applied to players of any level. Additionally, the model seeks to predict common mistakes made by players at this level, for which an auxiliary model specializing in this task has been implemented.

Ultimately, the model achieves a 44.41 % accuracy in move prediction, comparable to the results of other state-of-the-art engines while using a smaller amount of data. With these results, a bot named BlackBishop1700 has been created on the Lichess website, from which the data was extracted, allowing any user to play against it. This bot implementation required the use of the website's API and some adjustments to the chess engine to provide a satisfactory user experience. The website allows for post-game analysis of the moves played by both the player and the machine, thereby complementing the purpose of this project. We can conclude that human-like engines still bring many opportunities for improvement and ways of application and integration in other tools.

Introducción

1

Desde que los motores de ajedrez superaron a los humanos en su capacidad de juego, con DeepBlue¹ en particular, estos se han convertido en herramientas cotidianas en el proceso de análisis de las partidas de los grandes maestros o jugadores avanzados. Es muy común que en las retransmisiones oficiales de estas partidas, se indique en todo momento la evaluación que da la máquina a la posición en juego, comunicando fácilmente al espectador quién va ganando. Además, esta evaluación permite a los comentaristas explicar el resultado final en la hipótesis de distintas líneas de juego sin temor a interpretar incorrectamente las posiciones. Incluso, lo más común es que los jugadores avanzados analicen sus propias partidas junto con estas herramientas para entender dónde se han podido equivocar, u obtener nuevas ideas para jugar en las posiciones alcanzadas.

Por tanto, los motores de ajedrez son muy útiles para entender o analizar partidas y poder aprender y mejorar de ellas, tanto para grandes maestros como para un público novel.

1.1. Planteamiento del problema

Para jugadores noveles los motores presentan un grave problema que dificulta el aprendizaje. Los motores, en su mayoría, ofrecen prácticamente siempre la mejor jugada, calculando a una profundidad imposible de alcanzar para ningún ser humano. Sin un conocimiento previo del juego más profundo y de conceptos tales como las estructuras de peones o el desarrollo de las piezas, estos jugadores principiantes no son capaces de racionalizar por qué un motor prefiere unas jugadas sobre otras.

Debido al auge del ajedrez de los últimos años propiciado por la inclusión de este juego en la cultura *pop*, mediante *series* e *influencers* en redes sociales, se han creado grandes comunidades de jugadores *online*. La mayoría de estos jugadores no tienen conocimientos profundos del juego, por lo que han necesitado de ayudas, guías o cursos para poder entrar en él. Así, la gran mayoría ha acabado jugando *online* en portales que permiten realizar autoevaluaciones de sus propias partidas una vez concluidas utilizando un motor de ajedrez. Estas autoevaluaciones presentan los problemas mencionados previamente para los jugadores más noveles, por lo que aún es necesario seguir trabajando para encontrar formas de enseñar a estos jugadores hasta que sean capaces de introducir los motores de juego estándar en su análisis.

¹DeepBlue: Motor de ajedrez desarrollado en 1996. Fue el primer motor en vencer en una partida al campeón mundial de ajedrez vigente. Aún así, Gary Kasparov fue capaz de vencer en otras 3 partidas posteriores.

Una posible solución a este problema es la inclusión de los llamados *human-like engines* como herramientas de enseñanza para la realización del análisis. Estos motores no buscan encontrar la mejor jugada posible en una posición dada, sino predecir el movimiento que haría un jugador humano. Para ello, el motor debe tener algún conocimiento previo del jugador. Lo más común es que el motor esté preparado para predecir las jugadas de jugadores de una cierta puntuación o *ELO*².

1.2. Justificación

Existen múltiples motivos para pensar que la IA tiene todavía muchas posibilidades de aplicación en el campo del ajedrez. Para empezar, al poder enseñar correctamente a un jugador, es clave tener unos conocimientos previos de este que permitan personalizar este proceso. El nivel del jugador es el dato principal y en el que se sustentan motores como Maia³, pero todavía es posible ahondar mucho más en la personalización. El incremento del uso de webs para jugar *online* permite que se almacene un histórico previo de las partidas del jugador, con las cuáles se podrían conocer datos como sus aperturas favoritas, fallos recurrentes que comete, principios del ajedrez que todavía no entiende, entre otros. Incluso se están empezando a ver motores que “imitan” a otros jugadores, principalmente a jugadores de alto nivel. Todos estos datos ya se utilizan actualmente en algunas de estas webs para recomendar al jugador cursos o lecciones *online*, pero no se han incluido en herramientas automáticas de aprendizaje. Que el motor de juego entienda al jugador que lo está utilizando le permitiría prever y destacar posibles problemas que puede encontrar en la partida, y enseñarle a resolverlos.

Gran parte del problema de las herramientas de recomendación de jugadas existentes es que no hay forma de saber cuáles son los motivos por los que el motor ha preferido un movimiento sobre otro. Por su funcionamiento basado en ramificación y poda de árboles, esta decisión se basa en situaciones concretas a largo plazo dentro de la partida, del orden de una o dos decenas de movimientos. Así no es cómo los humanos piensan y razonan en el ajedrez. Por otro lado, la solución no es tan simple como definir una profundidad menor de búsqueda, ya que eso llevaría a errores muy graves en la partida de una manera inconsistente. Los humanos somos capaces de hacer movimientos que sabemos que van a ser buenos a largo plazo, simplemente porque dan muchas posibilidades de juego a futuro o porque son muy *robustos*, sin tener que calcular todas las posiciones exactas. Es por ello por lo que existen las llamadas aperturas “trampa”, ya que en ellas se intenta que el jugador realice estos movimientos *robustos* y tomarlo desprevenido. Por tanto, es necesario aproximar el funcionamiento de los motores a un carácter más humano, que les permita recomendar jugadas que sean entendibles por las personas.

Además, el incremento en el uso de webs de ajedrez ha permitido que no sólo tengamos

²ELO: Sistema de puntuación utilizado para comparar el nivel de juego de dos jugadores. Fue adoptado en 1960 por la Federación Estadounidense de Ajedrez.

³Maia: Publicado en 2020 por Reid McIlroy-Young, Siddhartha Sen, Jon Kleinberg, Ashton Anderson. Es un motor de ajedrez que busca predecir el movimiento que haría un jugador de un cierto nivel en una posición específica.

datos de las partidas de grandes maestros, sino datos masivos de un gran número de jugadores con mucha variedad de niveles y estilos de juego. Todos estos datos pueden ser aprovechados para la creación de nuevos modelos de *Machine Learning* (ML), que entiendan cómo juegan los jugadores de ajedrez que no son expertos en el juego. Por ejemplo, en *Lichess*⁴, uno de los portales de ajedrez de referencia, hay en activo unas 40 mil partidas de ajedrez, sumando todas las categorías y controles de tiempo posible. Esta web presenta una API pública que permite obtener las partidas jugadas en torneos que se celebren en la web o de jugadores en concreto. Toda esta información puede permitir la implementación de motores *human-like* como los mencionados anteriormente con las posibles aplicaciones beneficiosas para jugadores noveles que eso conlleva.

Por último cabe destacar que los motores *human-like* utilizan arquitecturas y modelos de IA muy distintos a los motores estándar. Motores como *Stockfish*⁵ se basan en árboles de búsqueda ponderados para encontrar la mejor jugada en cada posición, junto con técnicas de aprendizaje por refuerzo para calcular esas ponderaciones. Los motores *human-like* se basan en arquitecturas como redes neuronales convolucionales y rara vez hacen uso de algoritmos de búsqueda. Por tanto, hay un gran número de avances en relación a la arquitectura de estas redes que todavía pueden ser aplicables a estas soluciones. Además, al no basarse en algoritmos de búsqueda, eludirían el problema de depender de la capacidad de cálculo para su funcionamiento, permitiendo recomendar jugadas mucho más comprensibles.

1.3. Objetivos

Los objetivos a cumplir en este trabajo son:

1. Estudiar y analizar las características y funcionalidades de los motores *human-like* actuales. Es necesario comprender completamente su funcionamiento para poder adaptarlas e incluirlas en otras plataformas.
2. Recopilar datos de jugadores noveles y más avanzados. Estos datos deberán ser transformados según las necesidades de los modelos que se vayan a desarrollar.
3. Diseñar modelos y entrenarlos con los datos recopilados para la implementación de motores *human-like*.
4. Diseño de un algoritmo que permita el uso del modelo con herramientas de juego de ajedrez.
5. Estudio de rendimiento y resultados. Se medirá principalmente la capacidad de predecir el movimiento más humano que el jugador vaya a realizar.

⁴Lichess: Portal web para jugar ajedrez online desde 2015. Se caracteriza por ser de código abierto y publicado bajo licencia AGPL.

⁵Stockfish: Publicada su primera versión en 2008, es el motor de ajedrez más exitoso durante la década de 2010. Publicado bajo licencia GPLv3, sigue siendo muy revelante debido a su constante actualización y a su inclusión en portales de ajedrez online

1.4. Alcance

Consideraremos todos estos objetivos como cumplidos al desarrollar modelos que permitan la implementación de motores *human-like* y se integren con alguna herramienta de juego de ajedrez. Estos modelos deberán predecir cuáles serán los movimientos que debería hacer el jugador en una posición, manteniendo la comprensibilidad de los mismos, sin depender de búsquedas de alta profundidad. Se deberá adaptar la salida del modelo desarrollado para crear la comunicación con la herramienta de ajedrez utilizando un protocolo estándar como UCI⁶. Durante el proceso de estudio y análisis se definirán las métricas a utilizar para realizar la evaluación de los motores y compararlos entre sí.

1.5. Estructura

La presente memoria consta de los siguientes apartados.

El capítulo 2 corresponde al Estado del Arte correspondiente a los motores de ajedrez, haciendo una introducción sobre la evolución histórica de los mismos para llegar a los actuales motores *human-like* y sus resultados. También se ponen en contexto las herramientas de juego de ajedrez actuales, necesarias para interactuar con estos motores.

En el capítulo 3 se sitúa el Marco Teórico, donde se asientan las bases técnicas sobre las que se fundamentan los motores de ajedrez y en concreto los *human-like*. Se desarrollará el conocimiento básico necesario sobre arquitecturas de Deep Learning y los distintos paradigmas de aprendizaje automático que debemos tener en cuenta. A su vez, se presentará el juego de ajedrez de manera formal, y se describirán características a tener en cuenta para comprender el desempeño de estos modelos. También, se describirán las notaciones más comunes de las jugadas de ajedrez, que permiten comunicar al ordenador la disposición de las piezas sobre el tablero.

El capítulo 4 trata sobre los Materiales y Métodos utilizados durante el desarrollo de este TFM, describiendo así de qué datos y arquitecturas se parten y cómo se ha decidido trabajar a lo largo de todo el trabajo.

El capítulo 5 describe el tratamiento de los datos realizado, así como el desarrollo de los motores *human-like* propios para este trabajo, junto con las iteraciones previas que han permitido perfeccionarlos. Se describen los análisis de resultados que se han realizado tras cada una de estas iteraciones, y los conocimientos adquiridos y aplicados en la siguiente iteración. Al final, se realiza una evaluación de los distintos modelos y se describe cómo se unen con las herramientas de juego gráficas para que sean utilizables por un usuario.

El capítulo 6 corresponde a las Conclusiones, Limitaciones y Líneas Futuras de este proyecto. Se resumen los resultados adquiridos durante la elaboración de este trabajo, así como los conocimientos adquiridos en el mismo, y se enumeran los problemas descubiertos y las posibles futuras mejoras.

⁶UCI: Interfaz Universal de Ajedrez. Permite la comunicación entre los motores de ajedrez y las interfaces de usuario

Estado del Arte

2

Para entender los distintos acercamientos a la automatización del juego del ajedrez debemos entender las permutaciones que se dan en una partida. Cada movimiento de ambos jugadores tiene unas 10^3 posibilidades. Por tanto, en una partida de unos 40 movimientos, que suele ser su duración media, se deberían calcular 10^{120} variaciones. Si cada variación se calculara en un microsegundo, se tardaría 10^{90} años para terminar el cálculo y decidir cuál es el mejor movimiento.

Por esto, los motores de ajedrez han ido buscando distintas arquitecturas y soluciones para disminuir la complejidad del problema y obtener buenos movimientos en tiempos de computación razonables. No es sencillo comprender los últimos motores sin tener unas nociones de cómo han ido evolucionando durante las últimas dos décadas y de los avances en Inteligencia Artificial que se han realizado.

La principal característica de un motor de ajedrez es cómo realiza la evaluación de las posiciones en el juego. Cada motor necesita explorar las jugadas válidas en el tablero, por lo que necesita conocer al menos las reglas básicas del juego, y asociarles una puntuación para escoger cuál tiene más posibilidades de darle la victoria. A medida que se exploran los posibles movimiento y se evalúan, los motores generan un árbol de evaluación. Sobre este árbol se pueden aplicar distintas técnicas de búsqueda, como un algoritmo de minmax. Según cómo se calcula esta evaluación tenemos dos grandes tipos de motores.

2.1. Motores tradicionales

Un motor de ajedrez tradicional se basa en funciones predefinidas para evaluar la posición. Estas funciones suelen estar diseñadas por un grupo de expertos en el juego. Por ejemplo, una función muy sencilla sería contar cuantas piezas le quedan a cada jugador en una posición dada, siendo el jugador al que le queden más piezas el que tendría la ventaja. Este acercamiento fue el primero en aplicarse y ha ido haciéndose cada vez más complejo, con la inclusión de conceptos más avanzados. El primer referente en este campo es Deep Blue, desarrollado por [Campbell et al. \(2002\)](#), ya que es el motor al que se le atribuye



Figura 2.1: Garry Kasparov contra Deep Blue (1996). Fuente: [Worrall \(2018\)](#)

buye la primera victoria contra un Gran Maestro de ajedrez. Sin embargo, el motor más potente que ha venido utilizando este acercamiento ha sido Stockfish.

Stockfish, desarrollado inicialmente por [Romstad et al. \(2022\)](#) y documentado en su web, utiliza la poda alpha-beta como algoritmo de exploración del árbol de búsqueda. Esta poda se realiza con un valor dado de profundidad, permitiendo no explorar todas las posibles posiciones. El algoritmo de poda alpha-beta asume un juego óptimo por parte de ambos jugadores, por lo que permite no tener que explorar (podar) gran parte del árbol. A esta técnica se suma la aplicación de una serie de heurísticos que permiten podar aún más el árbol.

2.2. Motores basados en redes neuronales

Con la mejora de las técnicas de *Deep Learning* en campos como el aprendizaje supervisado y el aprendizaje por refuerzo, a finales de la década de los 2010 empezaron a aparecer modelos competitivos que basaban su función de evaluación en estas arquitecturas. Estas funciones eran aprendidas por el propio motor en vez de calculadas por un grupo de expertos. AlphaZero, desarrollado por [Silver et al. \(2017\)](#), fue uno de los modelos pioneros en este aspecto, ya que venció en numerosas partidas a Stockfish en 2018 sin ser vencido ninguna vez. Para calcular la función de evaluación, el motor utiliza una CNN (*Convolutional Neural Network*) y realiza un entrenamiento jugando contra sí mismo, únicamente conociendo los movimientos legales del juego. Este entrenamiento se realizó durante 9 horas, en las cuales se realizaron 700.000 *steps* de entrenamiento, aunque para vencer a Stockfish únicamente hicieron falta 4 horas de entrenamiento, unos 300.000 *steps*.

El sucesor de este acercamiento planteado por AlphaZero es Leela Chess Zero (Lc0), desarrollado inicialmente por [Linscott \(2018\)](#), un motor también basado en CNNs y aprendizaje por refuerzo jugando contra sí mismo. Su principal diferencia es la inclusión de las llamadas *Squeeze-and-Excitation Networks*, las cuales mejoran las CNNs tradicionales permitiendo que cada canal convolucional se pondere de manera distinta. Lc0 utiliza, en vez de un algoritmo de poda alpha-beta como Stockfish, un árbol de búsqueda de Monte Carlo (MCTS). Esto se debe a que la función de evaluación aprendida por el propio modelo en un inicio no es tan precisa como una función clásica, por lo que la heurística que brinda el algoritmo de Monte Carlo permite compensar esta imprecisión y obtener mucho mejores resultados.

A partir de este momento hubo un cambio en el paradigma de los motores de ajedrez. En la versión de 2020 de Stockfish 12 se sustituyó la función de evaluación clásica por una NNUE, *Efficiently Updatable Neural Network*, la cuál era más lenta de evaluar pero compensaba con su elevada precisión. Actualmente, Stockfish 16 se considera el mejor motor de ajedrez del mundo, el cual se basa en mejoras iterativas de la arquitectura propuesta en 2020. Este modelo no se basa en un entrenamiento por refuerzo como Lc0, sino en un aprendizaje sobre bases de datos de jugadores reales y otros motores de ajedrez.

Otros paradigmas en auge durante los últimos años incluyen los motores basados en modelos del lenguaje. Ya que las posiciones que se alcanzan en una partida de ajedrez siguen

una notación específica, ésta se puede utilizar como *input* para el modelo, al igual que cuando se procesan textos. Estos motores no necesitan conocer previamente las reglas del juego, ya que no realizan ningún tipo de búsqueda, sino que tokenizan los movimientos como si fuesen su vocabulario e intentan predecir la siguiente jugada. Una desventaja de esto es que pueden proponer realizar jugadas que no sean válidas en la posición en juego, por lo que es importante minimizar estos errores. En concreto, las recientes arquitecturas basadas en transformadores están empezando a tener unos resultados superiores a los de AlphaZero utilizando MCTS. Sin embargo, no está claro que se pueda llegar a igualar el resultado de motores como Stockfish o Lc0 aunque se sigan entrenando con más datos de mejor calidad. Aún así, utilizando *Large Language Models* (LLMs) como Chat-GPT se están consiguiendo unos buenos resultados aunque no estén específicamente entrenados para esta tarea. Como estudia [Toshniwal et al. \(2022\)](#), el juego de ajedrez es un escenario perfecto para evaluar la capacidad de estos modelos del lenguaje de mantener estados concretos sobre los que elaborar sus respuestas. Este planteamiento es muy interesante, ya que gran parte de las mejoras sobre este tipo de modelos que se han realizado estos últimos años buscan ampliar la "memoria" de estos durante una misma conversación.

2.3. Human-like engines

Los *human-like engines* son motores de ajedrez que no buscan optimizar su capacidad de juego como los motores previamente mencionados. Estos, por otro lado, buscan ser capaces de predecir el movimiento que haría en un jugador humano en una posición dada. Por tanto, el objetivo último de estos modelos es imitar el comportamiento humano. El problema principal al que se enfrentan estos motores es a la diversidad de jugadas que los humanos pueden llegar a realizar en una misma situación. Cada persona, dadas su condiciones particulares como su nivel, estilo de juego o personalidad, se decanta por unas estrategias u otras en una partida. Para reducir el impacto de este problema, los *human-like engines* necesitan conocer estas cualidades para poder realizar una buena predicción.

El principal *human-like engine* actualmente es Maia. Maia, publicado en 2020 y descrito en [McIlroy-Young et al. \(2020\)](#), parte de la arquitectura de AlphaZero, pero adaptándose para poder entrenarse con partidas de jugadores reales en vez de jugando contra sí mismo. Estos datos de entrenamiento están clasificados según el nivel de ELO del jugador, entrenando un modelo distinto para cada nivel. Al igual que los motores basados en modelos del lenguaje, Maia no realiza ningún tipo de búsqueda en árbol. Además de este modelo principal, Maia contiene un segundo modelo entrenado expresamente para predecir si el jugador va a cometer un error, que debe ser capaz de detectar jugadas que puedan engañar la propia evaluación de un jugador real. Mediante estos dos modelos, Maia obtiene unos resultados mucho mejores que los atribuidos a versiones adaptadas de Stockfish y Leela, sobre todo en jugadores más noveles. Dada la heterogeneidad de los propios jugadores, es una tarea difícil obtener unos buenos resultados para grupos grandes de jugadores, pero aún así se demuestra que aún se puede innovar en este campo.

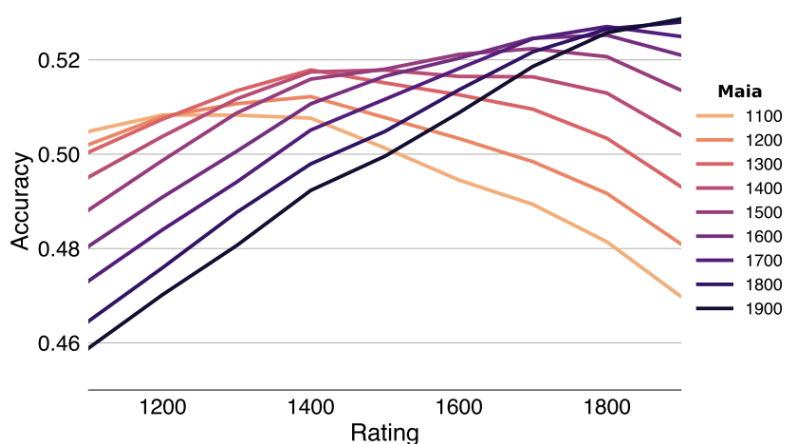


Figura 2.2: Accuracy obtenida por las versiones de Maia según ELO. Fuente: [McIlroy-Young et al. \(2020\)](#)

Los últimos avances van en la dirección del aprendizaje personal, es decir, de que los *human-like engines* sean capaces de imitar a jugadores concretos. Esto evita el problema mencionado de la heterogeneidad de los jugadores, pero abre nuevos frentes problemáticos. Para que estos motores sean útiles para la mayoría del público, no deben necesitar muchas partidas del individuo para poder amoldarse a su forma de jugar. Ya en 2022 se publicó una versión de Maia pensada para su uso personalizado, descrita por sus mismos autores en [McIlroy-Young et al. \(2022\)](#). Aunque el número de partidas que necesita es muy alto, del orden de varias decenas de miles, consigue obtener una mayor precisión que el modelo de Maia más cercano a la puntuación del jugador. Por tanto, se demuestra el potencial de los modelos de aprendizaje automático actuales para predecir e imitar el comportamiento humano.

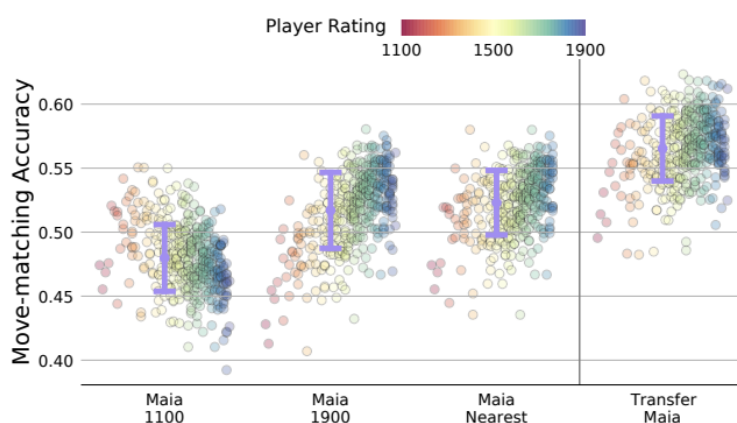


Figura 2.3: Accuracy obtenida para distintas versiones de Maia. Transfer Maia corresponde al modelo personalizado para un jugador. Fuente: [McIlroy-Young et al. \(2022\)](#)

2.4. Herramientas de juego de ajedrez

Una vez se tiene un motor de ajedrez capaz de calcular jugadas, es necesario utilizar una herramienta de juego que realice la propia simulación del tablero de juego. Aunque es posible utilizar una consola de comandos para ello, en la actualidad existen multitud de herramientas con interfaz de usuario que permiten a un jugador humano interactuar con las piezas del tablero cómodamente. Existe una gran variedad de herramientas, tanto aplicaciones de escritorio como web. La aplicación comercial más popular es ChessBase, la cual está muy enfocada en el análisis de bases de datos de ajedrez. Esta aplicación permite utilizar motores de juego personalizados para realizar el análisis de las partidas. Alternativas de código abierto a ChessBase son, por ejemplo, Arena Chess GUI o PyChess. Estos no incluyen motores tan potentes como las herramientas comerciales, pero presentan todas las funcionalidades útiles para integrar motores de ajedrez propios.



Figura 2.4: Interfaz Gráfica de Arena Chess

Es muy importante destacar también las herramientas web para jugar online. Las dos principales herramientas de estas características son Chess.com y Lichess.com. Estos sitios webs son utilizados diariamente por cientos de miles de personas para jugar partidas, pero además tienen disponibles herramientas de análisis utilizando distintos motores, cursos para aprender a jugar, puzzles con dificultad incremental, retransmisión de partidas en directo e históricos de las partidas de los jugadores de la plataforma. Incluso, Lichess tiene una API pública para obtener mucha de esta información, y permite la creación de perfiles para motores de ajedrez para poder analizar su rendimiento. Esto hace que Lichess sea una herramienta muy útil en el desarrollo de este TFM.

Marco teórico

3

A continuación se plantean los principios teóricos y los conceptos técnicos en los que se basa el desarrollo de este proyecto. Se tratarán las bases de *Machine Learning* y los distintos paradigmas que se presentan. Luego, se presentarán distintas arquitecturas basadas en redes neuronales profundas (*Deep Learning*). Con estos conceptos, se concretará en la arquitectura típica de un motor *human-like*. Por último, se darán algunas nociones de ajedrez necesarias durante el desarrollo del proyecto, como son las distintas notaciones utilizadas, o el sistema ELO.

3.1. Machine Learning y paradigmas de aprendizaje

El *Machine Learning* según [Springer \(1996\)](#) es un campo específico de la Inteligencia Artificial en el cual se desarrollan algoritmos estadísticos que aprenden de un conjunto de datos y obtienen una solución general sin que esta tenga que estar descrita explícitamente. Este campo, con el paso de los años, se ha ido diversificando y especializando en distintas áreas, que se categorizan principalmente atendiendo al proceso de aprendizaje que siguen sus algoritmos. Los 3 principales paradigmas son los siguientes:

1. Aprendizaje supervisado: Estos algoritmos procesan datos de entrenamiento que están ligados a la salida esperada tras su procesamiento, denominándolos como datos etiquetados. Estos modelos suelen estar orientados a tareas de regresión o clasificación de datos similares a los utilizados durante el entrenamiento. Los motores como Maia se consideran parte de este paradigma, ya que necesitan de partidas etiquetadas con el movimiento esperado para su entrenamiento.

2. Aprendizaje no supervisado: Estos algoritmos son capaces de extraer conocimiento de los datos sin que estos estén etiquetados. Algunos de los algoritmos clásicos de este paradigma son los algoritmos de *clustering*, los algoritmos de reducción de dimensionalidad o los *autoencoders*.

3. Aprendizaje por refuerzo: Este paradigma se basa en desarrollar un entorno con el que uno o varios agentes tienen que interactuar para resolver un problema. Estos agentes suelen entrenarse hasta que dominan las reglas que les impone el entorno. Son muy utilizados en entornos de juegos virtuales por lo estricto de sus reglas. *AlphaZero* es un buen ejemplo de

este paradigma, ya que aprende a jugar al ajedrez únicamente jugando contra sí mismo, sin ninguna referencia de partidas externas.

Además de estos 3 paradigmas principales, existen otros muchos intermedios, como pueden ser el Aprendizaje semi-supervisado o el Aprendizaje auto-supervisado. Por otro lado, dentro de los algoritmos de *Machine Learning*, debemos diferenciar los llamados algoritmos de *Deep Learning*.

3.2. Deep Learning

El *Deep Learning* no hace referencia al tipo de aprendizaje que lleva a cabo el algoritmo, sino al uso de redes neuronales profundas. Estas redes neuronales contienen una gran cantidad de neuronas que permiten modelar funciones no lineales mediante el aprendizaje de una serie de pesos. La actualización de estos pesos se basa en el algoritmo del descenso del gradiente y el *backpropagation*, lo que posibilita que, a partir de una función de pérdidas, el modelo sea capaz de acercarse a un mínimo, ya sea local o global, de la solución.

Este acercamiento es en gran parte empírico, y requiere del ajuste de una gran cantidad de hiperparámetros. La principal ventaja que aportan estos modelos es que presentan unas soluciones y arquitecturas capaces de resolver problemas muy diversos, ya que aceptan tipos de datos de dimensionalidades y características muy heterogeneas. Entre estas arquitecturas de *Deep Learning* destacan el MLP (*Multi Layer Perceptron*), la CNN (*Convolutional Neural Networks*), las RNN (*Recurrent Neural Networks*) o los LLM (*Large Language Models*). Estas redes son capaces de realizar tareas de los 3 paradigmas descritos anteriormente. A continuación detallamos las arquitecturas más importantes para este trabajo.

3.2.1. MLP

El *Multi Layer Perceptron* es la principal red neuronal utilizada para procesar datos tabulares o de características independientes entre sí. Suelen estar formados por las denominadas capas densas o *fully-connected*, en las cuales la salida de una neurona es parte de la entrada de las neuronas de la siguiente capa. La Figura 3.1 representa esquemáticamente las capas densas.

Estas neuronas asignan un peso a cada una de sus entradas, los cuáles se actualizan tras cada paso de entrenamiento gracias a la aplicación del Descenso del Gradiente. A su vez, cada neurona define una función de activación, la cuál permite mantener las salidas de las neuronas en unos rangos conocidos. Las funciones de activación más comunes son las funciones sigmoide, tangente hiperbólica y ReLU, junto con su variante Leaky ReLU, pero existe una gran variedad dependiendo de las necesidades de la red y los datos. La Figura 3.2 muestra las principales funciones de activación.

Cada función tiene ventajas e inconvenientes y es trabajo del desarrollador de la red el elegir las funciones de activación que mejor se ajusten al problema a resolver.

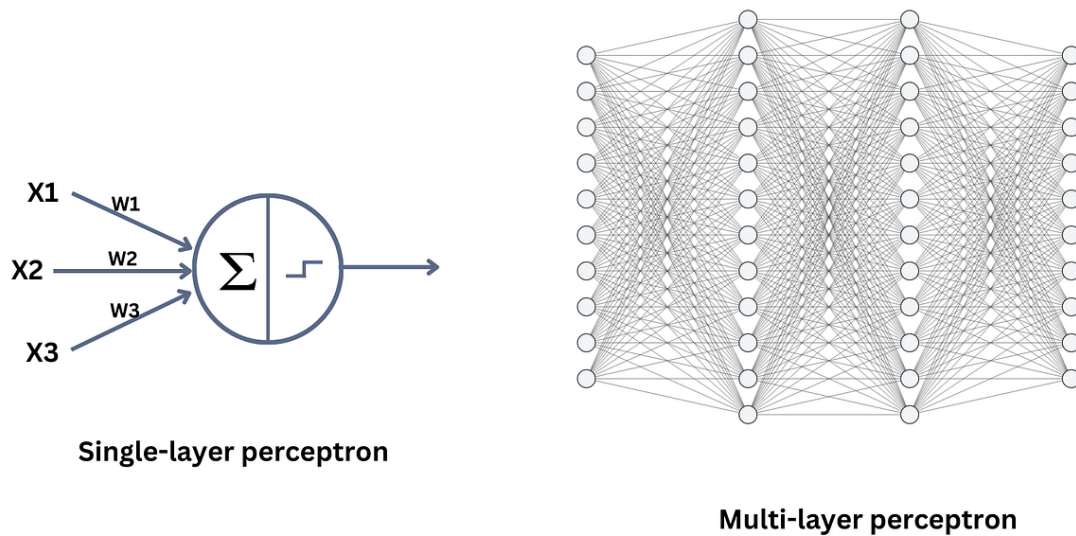


Figura 3.1: Representación de una neurona en un MLP. Fuente: [Amanatullah \(2023\)](#)

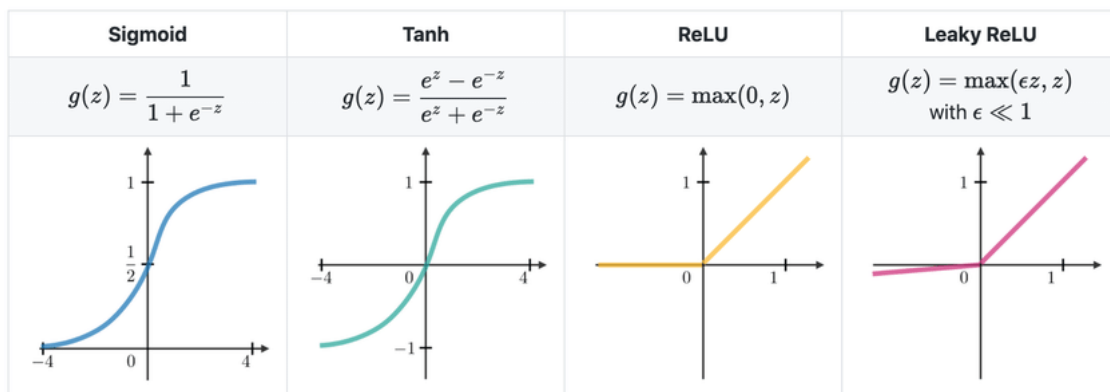


Figura 3.2: Funciones de activación más comunes. Fuente: [Loaiza \(2020\)](#)

3.2.2. CNN

Las *Convolutional Neural Networks* utilizan la operación convolución sobre unos datos espaciales para obtener conocimiento de los mismos y obteniendo una salida que también se puede considerar espacial. Esta convolución se trata de superponer un filtro o *kernel* sobre la imagen e ir realizando unas operaciones matemáticas utilizando los operadores aprendidos por la red.

Estas capas convolucionales se pueden ir concatenando para, por ejemplo, reducir la dimensionalidad de la imagen y así obtener un espacio latente que permite a otras redes procesar la información, como por ejemplo en los Autoencoders Variacionales. Las CNN son muy utilizadas también en las GAN (*Generative Adversarial Networks*), las cuáles permiten generar imágenes nuevas entrenándose con otros conjuntos de imágenes similares a las esperadas. Aunque lo más común es la aplicación sobre datos bidimensionales, como las imágenes, tam-

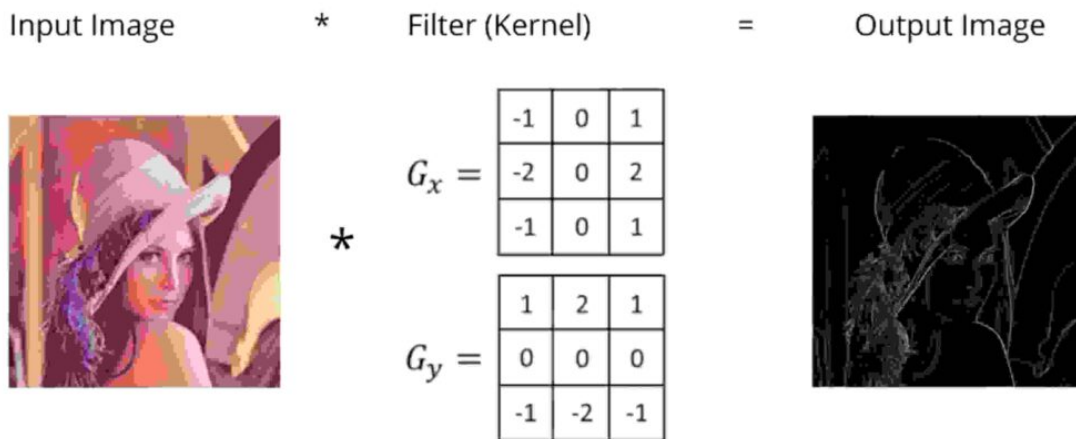


Figura 3.3: Ejemplo de kernel para obtener bordes de una imagen. Fuente: [Kumar \(2020\)](#)

bién es posible definir las y aplicarlas sobre una dimensionalidad mayor.

3.2.3. Conceptos fundamentales del proceso de entrenamiento

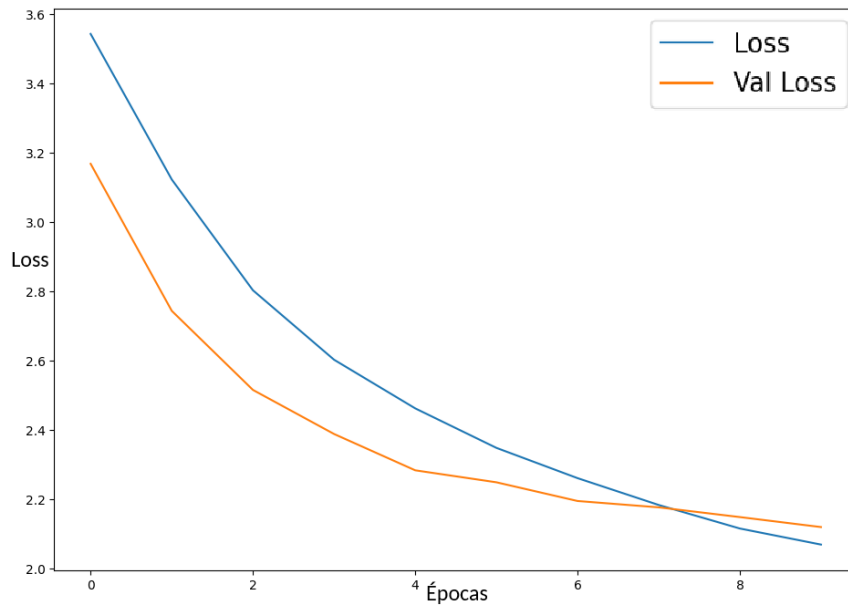
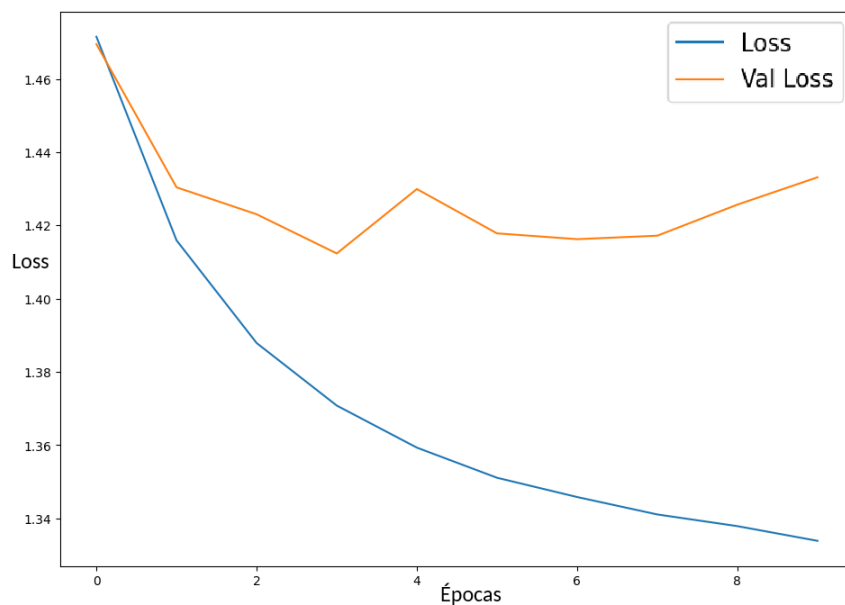
El proceso de entrenamiento de una red es un proceso empírico, que requiere de la comprensión y ajuste de una serie de hiperparámetros para su correcto desarrollo. Aún así, el campo del *Deep Learning* ya es capaz de acotar y comprender los efectos que tienen algunos de estos hiperparámetros que son muy comunes.

Para empezar, los datos suelen dividirse en 3 grupos, datos de entrenamiento, validación y pruebas. Estos 3 subconjuntos deben ser disjuntos, pero presentar una distribución lo más similar posibles, tanto en las características o *features* sobre las que se quiere realizar el aprendizaje como sobre la etiqueta final, si es que se trata de un entrenamiento supervisado. Para realizar el entrenamiento, es clave definir una función de pérdidas (*loss*), la cual sea capaz de definir numéricamente el desempeño de la red. El objetivo de la red debe ser minimizar el valor de esta función.

Al introducir estos datos en la red, se suelen agrupar en *batches*. El tamaño de un *batch* es un hiperparámetro que condiciona la velocidad a la que la red es capaz de generalizar el conocimiento adquirido. Un tamaño de *batch* alto suele llevar a una convergencia del modelo más rápida y lo hace más robusto a anomalías en los datos de entrenamiento.

Por otro lado, el valor de *learning rate* modela lo rápido que aprende la red en cada paso de entrenamiento (*step*). Un valor alto permite a la red acercarse rápidamente a mínimos en la función de pérdidas definida, pero no es capaz de obtener una solución precisa. A su vez, un valor bajo sí es capaz de minimizar mucho la función de pérdidas, pero es muy probable que no sepa encontrar el mínimo global. Debido a esta dualidad, es muy común que los procesos de entrenamiento comiencen con un *learning rate* alto, para encontrar el mínimo global rápidamente, y luego se vaya decrementando para precisar la respuesta.

Otro aspecto importante durante el entrenamiento de la red es durante cuánto tiempo se entrena la red. Una red que se haya entrenado poco no es capaz de extraer todo el conocimiento que podría de los datos de entrenamiento. Este fenómeno se denomina *underfitting*. Por el contrario, una red que se haya entrenado demasiado es capaz de modelar los datos de entrenamiento individuales, perdiendo capacidad de generalización durante el proceso. Para visualizar este proceso, se suelen representar las curvas de la función de pérdidas sobre los datos de entrenamiento y validación.

Figura 3.4: Curvas de pérdida *underfitted*Figura 3.5: Curvas de pérdida *overfitted*

Como podemos ver en las figuras anteriores, una curva *underfitted* todavía no ha termi-

nado de converger (cuando la red converge presenta una curva que tiende a ser horizontal, especialmente en validación), mientras que una *overfitted* ha comenzado a perder valor de generalización (el valor de pérdidas para validación ha empezado a aumentar) tras cada época de entrenamiento. Una época no es más que el procesamiento de los datos de entrenamiento en batches. Lo que se busca idealmente es que ambas curvas, entrenamiento y validación, converjan a valores similares para preservar la capacidad de generalización de la red. La cantidad de datos de los que se dispone para realizar el entrenamiento tiene un gran impacto en estas curvas, ya que muy pocos datos llevan a un rápido *overfitting* de la red, mientras que muchos alargan el proceso de entrenamiento.

3.3. Arquitectura del human-like engine

A continuación examinaremos la arquitectura de la red Maia descrita en [McIlroy-Young et al. \(2020\)](#), ya que es la principal representante de los human-like engines. Maia se puede clasificar como una *Convolutional Neural Network*. Estas redes se suelen utilizar en el procesamiento de imágenes, debido a su capacidad de obtener conocimiento de las relaciones espaciales que tienen los datos de entrada. Por tanto, al procesar un tablero de ajedrez, es lógico que la disposición espacial de las piezas se tenga en cuenta en la red. El motor consta de dos redes distintas. Una primera red que se dedica a la predicción del siguiente movimiento a realizar, y una segunda red auxiliar que predice si el jugador del nivel que se está imitando es propenso a cometer un error en la posición dada.

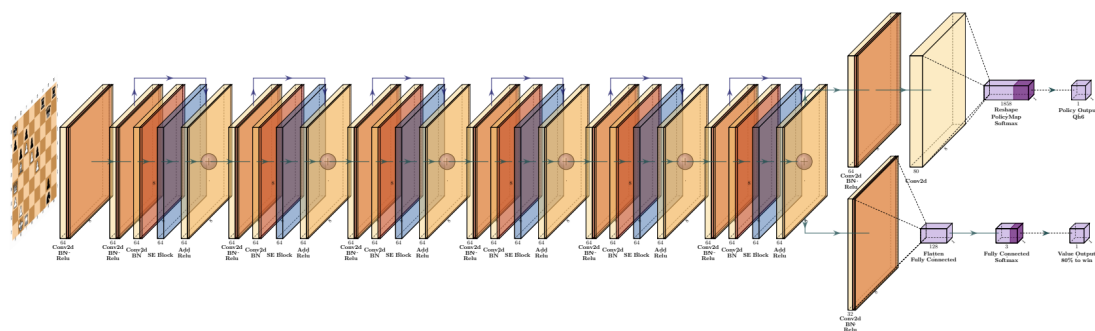


Figura 3.6: Arquitectura de red Maia para predicción del movimiento. Fuente: [McIlroy-Young et al. \(2020\)](#)

La primera red, encargada de predecir el movimiento, denominado *policy*, está compuesta por 6 bloques convolucionales, los cuáles cada uno están formados por 2 capas convolucionales con sus respectivas capas de *Batch Normalization*, una capa SE (*Squeeze-and-Excitation*) y una capa final que suma los *outputs* de la primera capa convolucional con la capa SE y aplica una función de activación ReLU. Las capas de *Batch Normalization* permiten que los valores de la red no se disparen, acercando la media de los *outputs* a 0 y la desviación estándar a 1. Las capas SE introducidas en [Hu et al. \(2019\)](#) permiten crear interdependencias entre los canales de la CNN con el objetivo de mejorar la representación del tablero de juego.

Por último, la red se divide en dos salidas distintas. La primera genera el nuevo movimiento a realizar en la posición dada, cambiando las dimensiones de la salida de la CNN, y utilizando una capa densa que sirva de *Policy Map* para seleccionar el movimiento. La función de activación *softmax* permite representar la salida de la red como una distribución de probabilidad sobre los posibles movimientos. La segunda salida utiliza dos capas densas con una función de activación *softmax* para obtener la probabilidad de victoria dada en la posición.

La red de predicción de error sigue una arquitectura bastante similar con 6 bloques convolucionales, y fue entrenada con los mismos datos que la red de *policy*. Sin embargo, utiliza unos valores menores en cuanto a número de *steps* ejecutados en el entrenamiento y valor de *learning rate* inicial.

3.4. Notación y comunicación de jugadas de ajedrez

Un tablero de ajedrez está formado por un tablero de 8x8 casillas, en las cuáles 2 jugadores, las Blancas y las Negras, tienen 16 piezas cada uno (8 peones, 2 torres, 2 caballos, 2 alfiles, una dama y un rey). El objetivo del juego es hacer jaque mate al rey rival, es decir, llegar a una posición en la que el rey enemigo esté siendo atacado por alguna pieza aliada y el jugador rival no pueda realizar ningún movimiento válido para salir de esta situación.



Figura 3.7: Posición inicial

Debido al gran número de movimientos posibles es importante que se defina un lenguaje común para comunicar jugadas de ajedrez y que éstas no lleven a equívoco. Para ello, el sistema de notación oficial y aceptado a nivel mundial para representar movimientos es el sistema de notación algebraica.

3.4.1. Notación Algebraica

Este sistema asigna a cada columna del tablero una letra minúscula y a cada fila un número, ambos comenzando desde la casilla inferior izquierda de las piezas blancas. Así tenemos

columnas nombradas de la *a* hasta la *h* y filas de la *1* a la *8*. Para designar una casilla concreta basta con concatenar una letra de columna y un número de fila. Por ejemplo, **b3**, hace referencia a la casilla que está en la segunda columna y en la tercera fila (Figura 3.8).

A su vez, las piezas tienen asignada una letra mayúscula. Estas letras suelen personalizarse para cada idioma, pero la notación más extendida es la inglesa, ya que es la más utilizada en herramientas de ajedrez. Así, **K**, **Q**, **R**, **B**, **N** hacen referencia al rey (*king*), dama (*queen*), torre (*rook*), alfil (*bishop*) y caballo (*knight*). Para representar un movimiento se concatena el nombre de la pieza con la casilla de destino. Por ejemplo, el movimiento desde la posición inicial del tablero de ajedrez **Nf3** representa que el caballo de **g1** se desplaza a la casilla **f3** (Figura 3.9). Si hubiese posibilidad de equívoco, cuando dos piezas iguales puedan desplazarse a la casilla destino, se añadirá la columna o la fila de la pieza que se quiera mover (**Ngf3** en el caso mencionado si hubiese ambigüedad). Para los movimientos de peón no es necesaria ninguna letra que los represente. El enroque es un movimiento especial que se denomina **O-O** si es enroque corto o **O-O-O** si es enroque largo, en el que el rey se mueve a la vez que la torre.



Figura 3.8: Casilla b3



Figura 3.9: Nf3

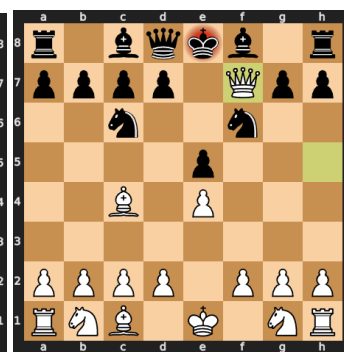


Figura 3.10: Qf7++

Si el movimiento captura una pieza del rival, se añade una *x* antes de la casilla destino (**Nxf2**). Si la captura la realiza un peón, se escribe la columna del peón antes de la *x*. **exd5** representa que el peón de la columna *e* captura la pieza de la casilla **d5**. A parte, si el movimiento hace *jaque* al rey rival, es decir, hay alguna pieza que podría capturarlo si fuese de nuevo su turno, se añade un *+* al final del movimiento (**Bb3+**). Si el jugador rival no tiene movimientos para hacer que el rey deje de estar en *jaque*, se considerará *jaque mate* y la partida habrá acabado. Esto se denota con *++* al final del movimiento (**Qf7++** en Figura 3.10). Los movimientos se enumeran de dos en dos (uno de blancas y uno de negras) para declarar cuántos movimientos han ocurrido (**1. e4 e5 2. d4 Nc6**).

3.4.2. Notación FEN

Otra notación que será importante comprender para este trabajo es la notación Forsyth-Edwards (FEN). Esta notación no describe movimientos como la notación algebraica, sino posiciones del tablero. Esto permite trabajar con posiciones descontextualizadas, sin necesidad de partir de la posición inicial y conocer todos los movimientos realizados. Esta notación se

compone de una única línea de texto, donde se representan cada fila del tablero entre separadores diagonales (/), empezando por la fila 8 hasta la 1. Dentro de cada fila, se disponen las piezas desde la columna 1 a la 8, con minúscula para Negras y mayúscula para Blancas. Si hay casillas vacías, se enumera cuántas casillas vacías seguidas hay en la fila.



Figura 3.11: Posición de jaque mate para blancas

La Figura 3.11 representa la posición de mate previamente mencionada, para la cual su notación FEN sería:

`r1bqkb1r/pppp1Qpp/2n2n2/4p3/2B1P3/8/PPPP1PPP/RNB1K1NR`

Si la analizamos, vemos que la notación señala que en la fila 8 de izquierda a derecha hay una torre negra, un hueco, un alfil negro, la dama negra, el rey negro, un alfil negro, otro hueco y otra torre negra. A su vez, vemos que la fila 3, que no tiene ninguna pieza, contiene un 8 en FEN, ya que hay 8 casillas vacías seguidas. Esta notación, por ejemplo, es la que se suele obtener al exportar una posición desde herramientas de ajedrez o portales *online* para que un motor las pueda analizar. A esta notación se le suele añadir al final el jugador activo en la posición (a cuál le toca jugar), los derechos de ambos jugadores a enroscarse y el movimiento por el que van dentro de la partida.

Estas dos notaciones son las más utilizadas para registrar partidas, siendo fácilmente legibles tanto para humanos como para máquinas. También existen variaciones importantes de estas notaciones, como la notación algebraica extendida, que denota siempre la casilla de origen del movimiento. Esta notación es la que se suele utilizar como protocolo de comunicación en los *Universal Chess Interface* (UCI), que nos permiten trasladar la salida de nuestro motor a una partida de ajedrez. Estos UCI se encargan de la capa de presentación del juego de ajedrez, pero además suelen contener tablas de aperturas, movimientos iniciales ya establecidos, además de otras funcionalidades para que el motor no tenga que cargar con esa información.

3.4.3. ELO

Otros aspectos importantes del ajedrez son el ELO y las distintas modalidades de juego. El ELO es un método matemático para el cálculo del nivel de un jugador. A grandes rasgos, cada vez que un jugador gana una partida contra otro en un torneo reglado, este consigue unos puntos que suma a su ELO actual. Cuanto más ELO tenga el jugador al que ha vencido, más puntos consigue. Por lo general, los puntos de ELO que gana un jugador son similares a los que pierde el otro. Los portales online han tomado esta idea para que cada jugador tenga su propio ELO dentro del portal. A su vez, la fuerza de los motores de ajedrez se suelen comparar en términos de ELO.

El ELO clasifica como profesionales a los jugadores con una puntuación mayor de 2000 puntos y a los grandes maestros por encima de 2500. También sitúa un límite superior teórico en los 3000. Aún así, a los motores de ajedrez se les suele atribuir un ELO mayor de 3000, ya que son capaces de ganar consistentemente a los mejores jugadores del mundo y a otros motores ya establecidos como super-humanos.

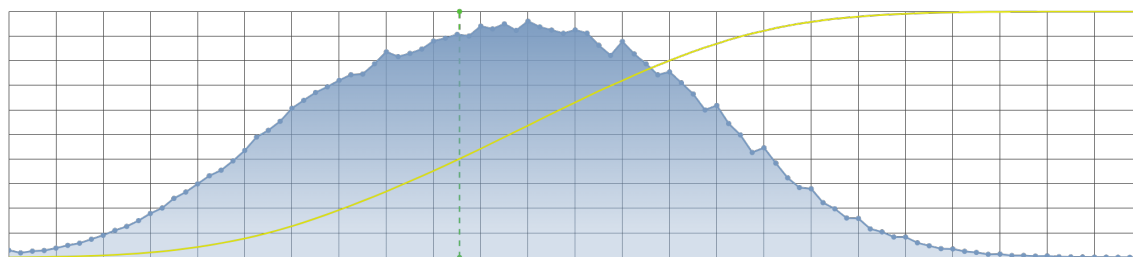


Figura 3.12: Distribución de jugadores por ELO en *Rapid* en Lichess.com. Fuente: [Lichess \(2024\)](#)

Por último, para poder valorar correctamente el nivel de juego de una partida es muy importante conocer la modalidad o *time-control* utilizado en la partida. El *time-control* denota el tiempo máximo que tiene cada jugador para realizar sus jugadas. Una partida Clásica suele dar 2 horas a cada jugador, sumando 1 hora más a cada uno si llegan al movimiento 40. Una partida Rápida da 15 minutos a cada jugador y suele sumar 5 segundos extra tras cada movimiento. Finalmente, una partida Bala da un minuto a cada jugador y puede no sumar nada de tiempo. Si a alguno de los jugadores se le acaba el tiempo, este pierde la partida, da igual la situación actual del tablero. Por tanto, la modalidad de juego que se esté jugando tiene un gran impacto sobre qué movimientos juegan los jugadores.

Metodología y materiales

4

En este capítulo se presentará la metodología CRISP-DM, aplicada durante el desarrollo de este proyecto. Además, se detallarán los datos utilizados para entrenar los distintos modelos de *Machine Learning* desarrollados, así como el *hardware* y *software* empleados.

4.1. Metodología

Para asegurar que se aplica un enfoque estructurado durante la ejecución de este proyecto, se aplicará la metodología de Ciencias de Datos CRISP-DM durante todo el desarrollo del mismo. Esta metodología, descrita en [Wirth y Hipp \(2000\)](#), define una serie de fases que componen la minería de datos misma, de manera cíclica. Como sus propios autores destacan, el proceso no termina con la fase de despliegue, sino que influye en el entendimiento del negocio, enriqueciendo futuros desarrollos. Las fases de esta metodología se resumen en el diagrama de la Figura 4.1.

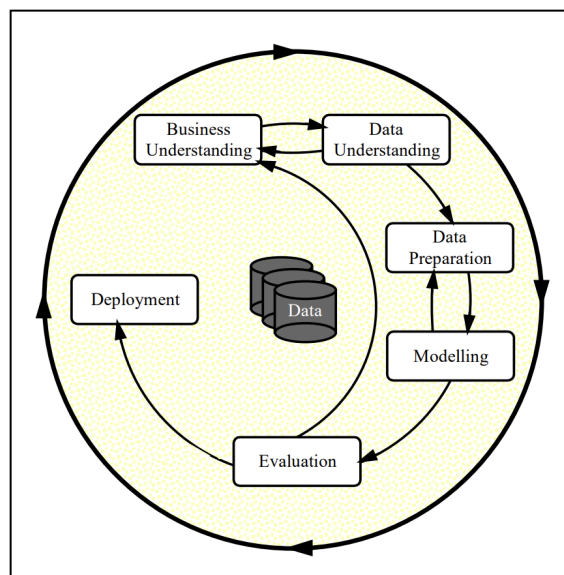


Figura 4.1: Diagrama de metodología CRISP-DM de [Wirth y Hipp \(2000\)](#)

Las fases de la metodología CRISP-DM son las siguientes:

1. **Entendimiento del negocio:** Antes de comenzar cualquier desarrollo es crucial conocer

los objetivos y el alcance del proyecto al que nos estamos enfrentando. Este entendimiento condicionará los datos que serán necesarios y los procedimientos para obtenerlos.

2. **Entendimiento de los datos:** Una vez se tiene un conjunto de datos (o *dataset*) con el que trabajar, es necesario conocer los datos de los que estamos partiendo. Esta fase incluye tareas como el conocimiento de los tipos de cada dato, los distintos valores que pueden tener, sus rangos, pero también las relaciones que tienen entre ellos. Estos datos, muchas veces, brindan nueva información sobre el negocio, por lo que pueden condicionar nuestro entendimiento del mismo.
3. **Preparación de los datos:** A partir del *dataset* seleccionado, se aplicarán técnicas de limpieza de datos y *feature engineering* que permitirán su uso posterior. Estas tareas incluyen selección de características, eliminación de valores nulos o anómalos, extracción de características derivadas de otras o adaptación de características a modelos, entre otros.
4. **Modelado:** En la Ciencia de Datos, el modelado hace referencia a la selección y desarrollo de modelos capaces de extraer la información necesaria para el negocio a partir de los datos obtenidos. Este proceso suele ser iterativo y empírico, y está fuertemente ligado a la preparación de los datos. Por tanto, los modelos seleccionados condicionarán la fase de preparación de los datos y viceversa. Además de su selección y desarrollo, los modelos deben ser optimizados, por ejemplo, mediante el ajuste de sus hiperparámetros.
5. **Evaluación:** La fase de evaluación consiste en definir y calcular distintas métricas que nos permitan saber si los modelos desarrollados son capaces de resolver el problema de negocio. Esta fase de definición no es baladí, ya que la selección de estas métricas debe tener en cuenta la naturaleza de los datos y cómo afectan al negocio. Finalmente, se debe definir un plan de acción en función de los resultados obtenidos, por ejemplo definiendo una eficacia mínima de resolución del problema y sus acciones en consecuencia de superarse o no.
6. **Despliegue:** Esta última fase del ciclo describe el proceso de despliegue de los modelos desarrollados, de manera que puedan ser utilizados por el cliente, o para su uso sobre un conjunto de datos final. También puede conllevar, por otro lado, la elaboración de informes para el cliente, si ya se ha venido trabajando con los datos finales durante el proceso. Estos informes deberán dar la solución al problema de negocio o, al menos, describir los resultados obtenidos en la evaluación de manera comprensible para equipos no expertos en el área de la Ciencia de Datos.

Como ya se ha mencionado, CRISP-DM será la metodología seguida durante el desarrollo de este TFM. Durante los capítulos 1, 2 y 3 hemos detallado el problema a resolver, definiendo el alcance y los principales objetivos. A su vez, hemos entendido los acercamientos actuales a resolver este problema y las herramientas teóricas que necesitaremos. Todos estos conocimientos constituyen la fase 1 de esta metodología, el entendimiento del negocio. Durante los

subsecuentes capítulos y apartados se describirán cómo se han desarrollado las siguientes fases en profundidad. A nivel general:

- La fase de entendimiento de los datos conllevará un estudio del tipo de datos que tenemos. En concreto, cómo están siendo representados los tableros de ajedrez y los movimientos en el mismo. A su vez, se necesitará conocer la distribución del *dataset* en función de las puntuaciones de los jugadores y la distribución de jugadas en todo el tablero, ya que es muy posible que no sean datos balanceados.
- En la fase de preparación se filtrarán y limpiarán los datos, eliminando posibles anomalías. A su vez, se deberá preparar la representación del tablero de ajedrez para su procesamiento por parte del modelo, por lo que también será necesario conocer qué modelos se pretenden desarrollar.
- La fase de modelado consistirá en proponer e implementar una nueva red, capaz de predecir jugadas de ajedrez. Esta red será entrenada con los datos reservados para el *dataset* de entrenamiento, procesados en el paso anterior. Además de la arquitectura de la red, esta tarea también comprende el ajuste de hiperparámetros.
- Durante la fase de evaluación, se deberá realizar un seguimiento de los resultados obtenidos durante el entrenamiento, mediante el estudio de las curvas de entrenamiento y la prueba con los datos de *test*. Una vez tengamos un modelo satisfactorio en este aspecto, se procederá a realizar una evaluación más concienzuda y se comparará con la de otros modelos del estado del arte que busquen resolver esta tarea.
- La fase de despliegue comprende la adaptación del modelo para su uso como motor de ajedrez. En concreto, se pretende implementar un *bot* de Lichess contra el que cualquier usuario de la web pueda jugar. Durante esta fase, también pueden surgir nuevos *insights*, por lo que puede que se necesitará estudiar su desempeño en el nuevo contexto y realizar cambios en base a ese nuevo conocimiento.

Tal y como se desarrollará más adelante, las fases de preparación de datos, modelado y evaluación, se desarrollarán de manera iterativa, realizando un incremento de la complejidad del problema a resolver en cada una. Así, los conocimientos que se irán obteniendo sobre el dominio del problema, propiciarán mejores resultados en las iteraciones posteriores. En concreto, se empezará planteando el problema como la predicción de qué pieza moverá el jugador, únicamente partiendo de jugadas de Blancas. El objetivo final será la predicción del movimiento completo, tanto para Blancas como para Negras. Este modelo final será el utilizado para la fase de evaluación más concienzuda y comparativa, y para la fase de despliegue.

4.2. Materiales

A continuación se detallan los materiales utilizados para el desarrollo de este proyecto. Esto incluye los datos recopilados y las herramientas *software*, además de las características

del *hardware* empleado.

4.2.1. Datos

Según nuestro entendimiento del negocio, para resolver el problema se desarrollarán modelos que intentarán predecir qué jugada hará un jugador de cierto nivel ante una posición dada. Debido a este propósito, el *dataset* debe cumplir las siguientes condiciones:

1. El *dataset* debe contener, como mínimo, representaciones de partidas de ajedrez en alguno de los formatos de texto estándar para su representación, relacionados con el nivel de los jugadores que están jugando la partida.
2. El *dataset* debe ser muy extenso y variado. Como se ha expuesto previamente, existe una gran cantidad de posiciones distintas en el juego, por lo que es importante que estén presentes en los datos. Además, hay que tener en cuenta que estamos prediciendo un comportamiento humano, por lo que ante un mismo *input* pueden existir múltiples *outputs* válidos. Es decir, distintos jugadores harán jugadas distintas en las mismas posiciones, por lo que se aumenta aún más la necesidad de una gran representación de jugadores distintos.
3. A partir de la representación de los datos que se utilice en el *dataset*, debe ser posible extraer la jugada que se ha realizado en la posición, ya que es la etiqueta que se pretende predecir. Las posiciones de juego sin un contexto de cómo actúa el jugador no son útiles, ya que no siempre se realizan las mismas jugadas.
4. El *dataset* deberá asociar algún tipo de evaluación de la calidad del movimiento, para poder predecir si el jugador va a cometer un fallo o no. Este punto no es estrictamente necesario, ya que siempre es posible realizar la evaluación mediante un motor de uso libre como *Stockfish*. Sin embargo, este proceso es muy lento y costoso computacionalmente, por lo que es preferible que la evaluación venga por defecto. Por suerte, hoy en día es una práctica común añadir esta información, ya que es la más utilizada por los motores que buscan maximizar la calidad de juego.
5. Sería deseable que el *dataset* incluyera más meta-información sobre la partida, aunque no sea imprescindible para el entrenamiento. Al menos, se buscará que contenga la modalidad de juego de la partida (*time control*) y los derechos de enroque de ambos jugadores.

Teniendo en cuenta todas estas condiciones, los datos que recoge y publica el portal web de ajedrez Lichess son los que más se acercan a las características deseadas. Lichess lleva publicando mensualmente desde 2013 un archivo de texto con la información sobre las partidas que se juegan en él. En total, Lichess ha publicado 5.837.020.176 partidas de muchos jugadores, por lo que cumple con los requisitos de extensión y variedad sobradamente. Sin

embargo, necesita de un procesamiento previo muy extenso, ya que contamos con una entrada por partida, lo cuál no es deseable. Por ello, para el entrenamiento del motor de juego Maia en [McIlroy-Young et al. \(2020\)](#), que también parte de los datos de Lichess, se realiza un procesamiento previo que permite un uso más cómodo de los mismos. Tras el desarrollo de Maia, estos datos procesados fueron liberados al público, por lo que son de uso libre.

Para empezar, los datos se encuentran en formato tabular, donde cada entrada representa las características correspondientes a una jugada en concreto y a la partida en la que se jugó. Se extraen 44 características para cada entrada.

- Las 11 primeras características hacen referencia a la partida. Contiene la modalidad de juego, el resultado final, los nombres de ambos jugadores, sus puntuaciones de ELO, el *time control*, las jugadas totales que dura, cómo se ha resuelto la partida (jaque mate, tablas, rey ahogado, repetición de movimientos o final de tiempo) y 3 columnas binarias que representan si han ganado las Blancas, las Negras o ha habido empate.
- Los siguientes 21 campos dan información sobre la jugada realizada. Los componen el número del movimiento en la partida, el movimiento en notación algebraica extendida y la evaluación de Lichess en varios formatos (puntuación total, variación con respecto al movimiento anterior, etc). Esta evaluación es un número decimal positivo si muestra ventaja para las Blancas y negativo si la ventaja es de Negras. También encontramos la información relativa al jugador activo en ese momento, como su ELO o su ratio de victorias. Por último encontramos información sobre el reloj en ese momento (cuánto tiempo le queda a cada jugador, cuánto ha pasado desde el último movimiento y si el movimiento se realizó con poco tiempo en el reloj).
- Al final, encontramos las *features* relativas al estado y disposición del tablero. La más importante es la disposición de las piezas, representada en una sola columna con una cadena de texto en notación FEN. Las últimas columnas muestran información sobre cuántas piezas de cada tipo le quedan a cada jugador, si en el tablero se está dando jaque y cuántos movimientos legales hay disponibles para el jugador activo.

Así, estos serán los datos de los que se partirá para realizar este trabajo. Se toman los datos de un mes de juego de Lichess adaptados por el equipo de Maia, correspondiente a las partidas jugadas en enero de 2019 únicamente, lo que forma un total de 135 millones de jugadas (o entradas en nuestro *dataset*) en un único archivo csv de 35GB. Hay que destacar que el modelo Maia fue entrenado con los datos desde abril de 2017 a diciembre de 2019. Ya desde un primer momento vemos claro que el manejo inteligente de los datos para aprovechar los recursos de los que disponemos será muy importante.

4.2.2. Herramientas y *hardware*

A continuación se enumeran las principales tecnologías utilizadas durante el desarrollo de este TFM, así como los recursos *hardware* de los que se dispone:

- El lenguaje de programación utilizado para el desarrollo será Python 3.10.14 en un entorno virtual de Anaconda. Esto incluye el uso de las librerías *Pandas* para la lectura de los datos tabulares, *Matplotlib* para la visualización, *Tensorflow* y *Scikit-learn* para el desarrollo de los modelos y *python-chess* para la simulación de partidas de ajedrez, entre otras.
- El IDE utilizado será Visual Studio Code.
- Se utilizará SQLite para almacenar y transformar los datos en gran volumen. No se considera que sea necesario el uso de herramientas de *Big Data* para trabajar con la cantidad de datos de la que disponemos.
- Se utilizará el portal Lichess como una de las plataformas de despliegue del motor desarrollado.
- La gestión de versiones se realizará mediante *Git*.

Se dispone de un PC con las siguientes características para el tratamiento de los datos y la ejecución del entrenamiento de los modelos:

- Sistema operativo: Windows 11.
- CPU: AMD Ryzen 5 5600X 6-Core Processor 3.70 GHz
- RAM: 16,0 GB
- GPU: Nvidia GeForce RTX 3060 12 GB GDDR6

Desarrollo y resultados

5

En este capítulo se presenta todo el desarrollo realizado durante el transcurso de este proyecto. Se divide en 4 apartados principales: estudio y preparación de los datos, desarrollo de los modelos de *Machine Learning*, evaluación de los resultados y despliegue del modelo. Debido al carácter iterativo de la metodología utilizada, la evaluación de los modelos es parte intrínseca del proceso de desarrollo, por lo que se presentarán varias evaluaciones a lo largo del capítulo que justifiquen las decisiones tomadas durante el mismo.

5.1. Estudio y preparación de los datos

Esta sección describe tanto el estudio inicial de los datos como los procesos aplicados sobre los mismos para obtención de las características finales a utilizar en nuestro modelo. Esta sección se corresponde con las fases 2 y 3 de la metodología CRIPS-DM. Se debe tener en cuenta que este proceso es desarrollado junto con la comprensión del negocio y el desarrollo de los modelos, por lo que se describirá cómo ha sido este progreso, haciendo especial énfasis en la estructura de datos utilizada finalmente.

5.1.1. Entendimiento y limpieza de los datos

Partimos de los datos proporcionados por el equipo de Maia en su web oficial, correspondiente a partidas jugadas en el portal web Lichess. En concreto, únicamente utilizaremos un mes de datos, e intentaremos optimizar todo lo posible los resultados sin añadir nuevas entradas. El *dataset* contiene un total de 135 millones de jugadas aproximadamente en formato tabular en un archivo .csv.

Para trabajar con ellos más cómodamente, se comienza por migrar los datos del archivo .csv a una base de datos relacional, en concreto SQLite. Almacenarlos en una base de datos SQL nos permitirá realizar consultas y transformaciones más fácil y rápidamente. Durante este primer paso, se comprobó que el gran número de entradas de la base de datos ralentizaba cualquier transformación que se quisiera realizar sobre los mismos, por lo que se comenzó un estudio inicial.

Ya que nuestro objetivo es predecir movimientos para un jugador de cierto nivel, se realizó una agrupación de los datos en función del ELO del jugador de Blancas en grupos de 100

puntos. Este nivel es similar al de su adversario. La distribución de nivel se puede visualizar en la siguiente Figura 5.1.

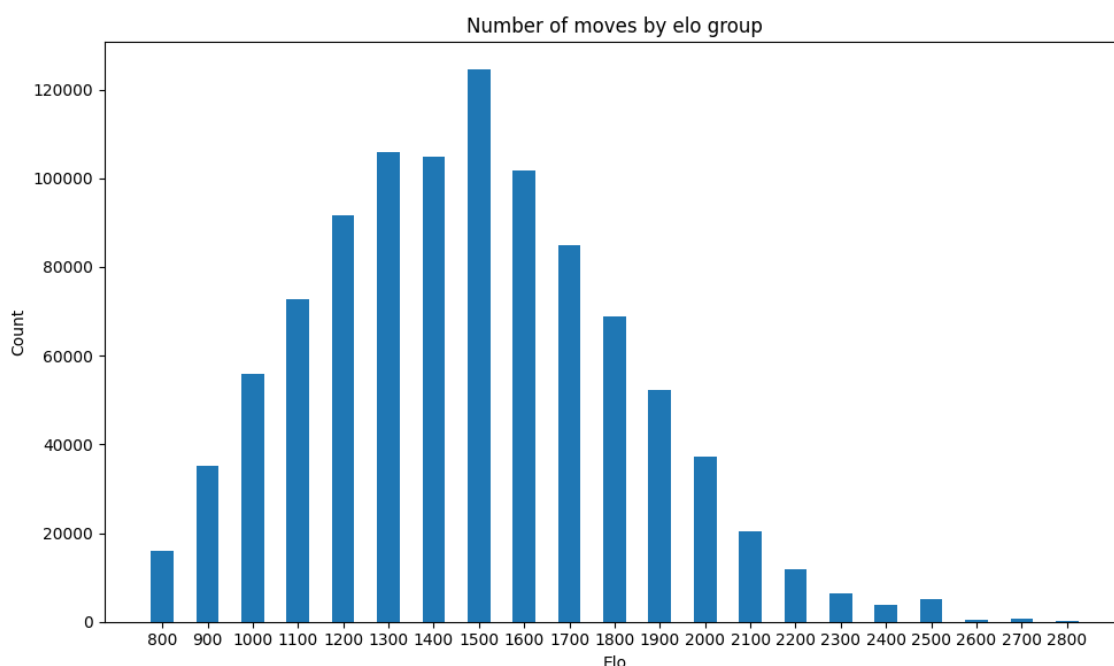


Figura 5.1: Agrupación de entradas por nivel para una muestra de 1 millón.

Como vemos, la distribución no es regular. Ya que la duración media de las partidas a distintos niveles es similar, podemos concluir que la distribución de nivel de los jugadores se puede ver representada mediante esta figura. Así, la gran mayoría de los jugadores se presentan en los niveles de juego intermedios, teniendo su máximo en 1500. Además, vemos que el número de jugadores con menos de 800 puntos o con más de 2300 es muy pequeño.

Para el desarrollo de los modelos se seleccionará uno de estos intervalos, ya que los movimientos realizados por un jugador de 1000 puntos no serán los mismos que los de uno de 2000. En este sentido, se debe tener en cuenta que jugadores de un menor nivel tienden a ser más erráticos e impredecibles, mientras que los de mayor nivel realizan jugadas más parecidas a las recomendadas por los motores de juego tradicionales. Esto lo podemos ver en [McIlroy-Young et al. \(2020\)](#), donde se calcula la capacidad de motores como *Stockfish* para la predicción de movimientos humanos.

Teniendo todo esto en cuenta, se decide limitarse a utilizar los datos de los jugadores de 1700 puntos. Creemos que son un grupo de jugadores con un buen balance entre número de datos recopilados y nivel de juego, sin ser profesionales ni del todo noveles. El propósito de este motor debe ser el jugar movimientos acertados en su mayoría, pero comprensibles por jugadores noveles (del rango de 800 puntos a 1200), por lo que imitar a un jugador de 1700 parece una opción correcta. Aún así, cualquier modelo que desarrollemos es fácilmente utilizable en otros rangos de niveles realizando un entrenamiento con datos de ese rango.

Otra característica importante a tener en cuenta para evaluar correctamente el nivel de juego de una partida es el *time-control* utilizado. La columna correspondiente a esta característica contiene 6 valores distintos, *UltraBullet*, *Bullet*, *Blitz*, *Rapid*, *Classical*, y *Correspondence*, ordenados de menor a mayor tiempo. De manera intuitiva, podemos razonar que un mismo jugador no realizará las mismas jugadas en una partida *Bullet*, con 1 minuto como máximo en el reloj para toda la partida, que en una *Classical*, con 30 minutos y 20 segundos extra por movimiento para la misma. Por tanto, se decide utilizar únicamente las modalidades centrales y más populares para el entrenamiento de los modelos, *Blitz* y *Rapid*, que corresponden a partidas entre 3 y 15 minutos. Así también podremos recoger una gran variedad de situaciones distintas. A su vez, también se decide eliminar las jugadas realizadas con poco tiempo en el reloj, ya que pueden ser muy distintas al resto. Estas jugadas podrían considerarse como datos anómalos y no queremos que influyan en la toma de decisiones de nuestro modelo.

Tras todos estos filtros aplicados a los datos iniciales, obtenemos una base de datos con un total de 9.581.626 jugadas. Aunque es un número mucho más manejable, esta base de datos sigue teniendo un número considerable de jugadas.

El siguiente paso es borrar las columnas que no son interesantes para la resolución del problema o que han perdido relevancia tras la aplicación de los filtros anteriores. Finalmente, únicamente nos quedamos con las siguientes columnas:

Columna	Tipo
board	String
is_check	Integer
move	String
is_blunder_cp	Integer

Tabla 5.1: Tabla “moves” que recoge las entradas del *dataset*.

- **move (String):** Movimiento realizado en esa jugada en notación algebraica extendida. Constituye la principal etiqueta a predecir por los modelos. Ejemplo: *e2e4*.
- **isBlunder (Integer):** Evaluación de *Stockfish* sobre si la jugada realizada es un fallo o no. Contiene valores binarios (0 o 1). Constituye nuestra segunda etiqueta a predecir.
- **board (String):** Distribución de las piezas en el tablero antes de realizar el movimiento descrito en la columna *move* en notación FEN en una cadena de texto. Además de esta información, la cadena también describe quién es el jugador activo, los derechos de enroque de cada jugador y el número de movimiento dentro de la partida. Estos datos serán los que el modelo deberá procesar para obtener el movimiento esperado y si es un error o no. Ejemplo: *rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1*.
- **isCheck (Integer):** Valor binario que describe si en la posición hay un jaque o no. Es un parámetro de entrada al modelo opcional que a priori podría ser útil.

Por tanto, contamos con dos características de entrada y dos etiquetas a predecir. A continuación se realizó un estudio de estas etiquetas.

Para empezar, la etiqueta *move* representa mediante la notación algebraica extendida la casilla donde se encuentra la pieza a mover y la posición a la que se desplaza. Esta notación es independiente de la pieza que se mueva, a diferencia de la notación algebraica estándar. Por tanto, se deduce que se asignará uno de 4096 valores distintos para esta etiqueta, el cuadrado del número de casillas del tablero. Sin embargo, muchas de estas combinatorias no son posibles, ya que no existen piezas capaces de realizar esos movimientos. Otros, incluso, serán demasiado poco frecuentes para que se encuentren representados en nuestro *dataset*. Así, se dispone de 1914 movimientos distintos (sin tener en cuenta la pieza utilizada) siguiendo las siguientes distribuciones:

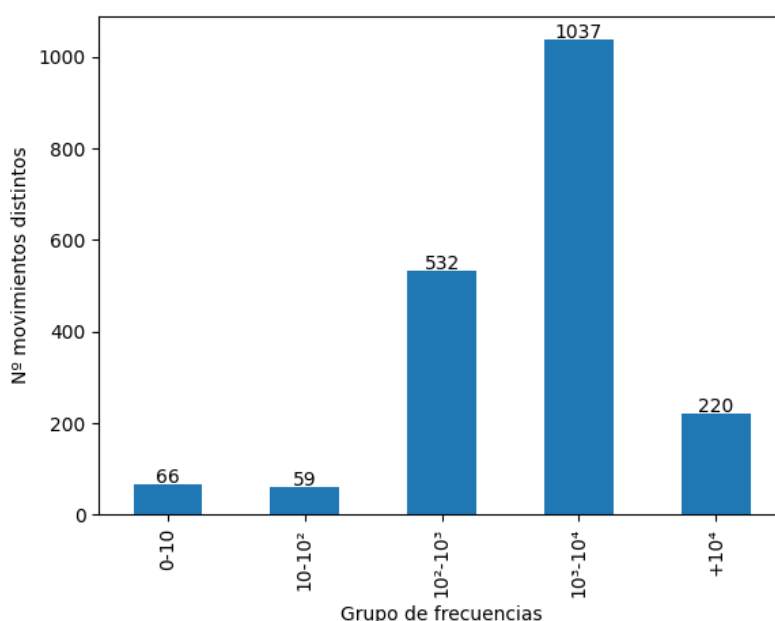


Figura 5.2: Distribución de frecuencia de movimientos

A la izquierda de la Figura 5.2 podemos observar cuántos movimientos presentan una frecuencia en nuestro *dataset* agrupados en conjuntos de potencia de diez. Vemos que la mayoría de las jugadas, un 54 % del total, aparecen entre 1.000 y 10.000 veces. Aún así, 220 jugadas, un 11 %, aparecen más de 100.000 veces. Esto se debe a que las jugadas comunes en las aperturas, donde se parte de un estado inicial conocido, y de los finales del juego, donde quedan pocas piezas, son mucho más frecuentes que las del medio juego.

Si nos fijamos ahora en la Figura 5.3, aquí se ordenan los movimientos de mayor a menor frecuencia. Esta gráfica nos permite visualizar fácilmente lo desbalanceado que está el *dataset*, algo previsible debido al gran número de movimientos posibles y a la naturaleza no determinista del juego humano. El movimiento jugado con más frecuencia, *g1f3*, es un movimiento que aparece 134.607 veces, por lo que constituye por si solo un 1,5 % del *dataset* (recordemos que partimos de 9.5 millones de jugadas). Los 10 movimientos más frecuentes superan las cien mil

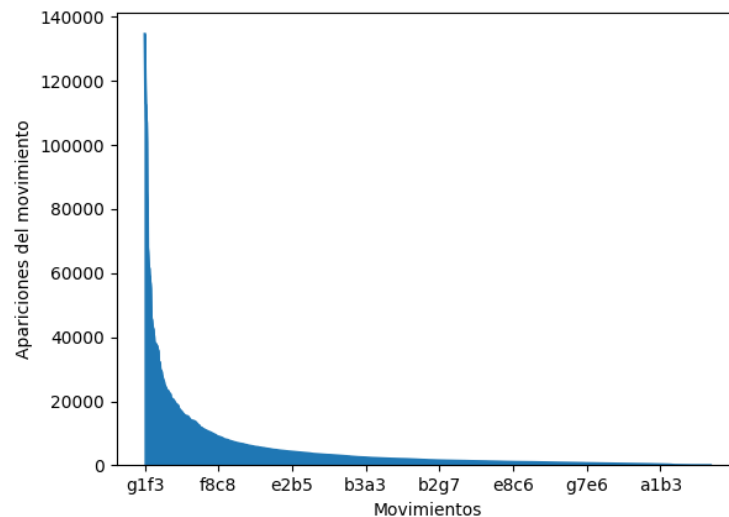
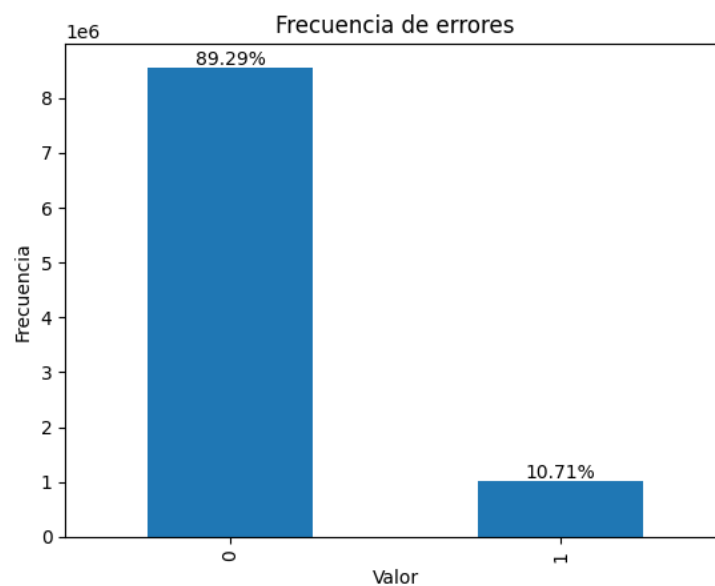


Figura 5.3: Movimientos ordenados por frecuencia

apariciones cada uno y suman un total de 1 millón de entradas, un 10 % del total. Realizando una suma acumulativa de estas frecuencias ordenando de mayor a menor, obtenemos que los primeros 138 movimientos ya suman más de un 50 % del total del *dataset*. Concluimos entonces que no sólo será necesario un análisis de la proporción de aciertos del modelo, sino también de que estos no sólo se limitan a un reducido número de movimientos.

En cuanto a la etiqueta *isBlunder*, la distribución de la misma también es muy desbalanceada, tal y como vemos en la Figura 5.4. Para evitar que este fenómeno tenga fuertes repercusiones en el rendimiento del modelo, será necesario utilizar métricas que permitan la monitorización correcta del progreso de entrenamiento del mismo, como se detalla posteriormente.

Figura 5.4: Frecuencia de errores según evaluación de *Stockfish*.

5.1.2. Extracción de características

Con el conocimiento de los datos del que disponemos podemos comenzar a realizar las transformaciones necesarias para su uso. Para empezar, la columna *board* contiene varios datos distintos que debemos separar en distintas columnas. Volviendo al ejemplo de tablero mostrado, tras la representación del tablero vemos la cadena de texto “w KQkq - 0 1”. Esta cadena es fácilmente tokenizable para obtener los valores que pueden ser útiles en un futuro. El primer carácter, *w* o *b* indica qué jugador es activo en la posición dada. Este dato se utilizará en la codificación del tablero. “KQkq” representa los derechos de ambos jugadores para enrocarse hacia el lado de rey o de reina. La presencia de cada letra representa uno de estos derechos. Por tanto, creamos 4 columnas binarias que almacenan esta información. Además, reservamos el valor final de la cadena, que representa el movimiento actual (*ply*), que también puede ser relevante en un futuro. Reservados estos valores, podemos borrar ese final de cadena y quedarnos únicamente con la representación del tablero.

rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR

Ya desde un comienzo, se ve que la representación del tablero estará fuertemente ligada al tipo de modelo que se utilice. Por ejemplo, si se deja los tableros como cadenas de texto tal y como están de inicio en la columna *board*, sería necesario aplicar modelos de procesamiento del lenguaje para extraer conocimiento de los mismos. Además, como no se trata de un lenguaje natural, los modelos actuales de *Natural Language Processing* pueden no ser fácilmente aplicables. Por ello, en un inicio se descarta su uso para la resolución de esta tarea.

La arquitectura de red que intuitivamente puede tener un buen desempeño es la Red Neuronal Convolucional (CNN). Esta arquitectura es ampliamente utilizada para el procesamiento de imágenes, debido a su capacidad de obtener información espacial. Si lo pensamos, un tablero de ajedrez es similar a una imagen digital de ocho por ocho píxeles. En vez de tener 3 canales de color, como en las imágenes RGB, podemos representar la presencia o ausencia de cada tipo de pieza mediante un “canal” distinto. También, se añade un canal extra que marque cuál es el jugador activo.

Así, se diseña un algoritmo que traduce las cadenas de texto a vectores de 3 dimensiones (8, 8, 7). Estas 7 componentes últimas corresponden con las 6 piezas distintas del ajedrez y al jugador activo. El jugador activo se codifica de manera binaria (0 o 1), mientras que, para las piezas un 1 presente en una de estas posiciones denota que una pieza blanca de ese tipo se encuentra en esa posición y un valor -1 denota la presencia de una pieza pieza negra. Un 0 marca la casilla vacía. Otra posibilidad podría ser representar las piezas negras con posiciones distintas a las blancas en la última dimensión, formando vectores *One-Hot Encoded* de (8, 8, 13). Sin embargo, como esto aumenta mucho la dispersión de los valores y el uso de memoria, se ha optado por utilizar únicamente 7 canales.

Por otro lado, el movimiento a predecir se divide en 3 columnas distintas, la columna original y dos columnas extra. Estas dos nuevas columnas almacenan las dos mitades del movimiento

por separado (posición inicial y posición final). Esta transformación se realiza para poder estudiar más cómodamente los datos de los que disponemos y poder, además, probar en un futuro distintos acercamientos a la resolución del problema. Por lo pronto, podemos visualizar la distribución de valores de estas nuevas etiquetas. Las figuras 5.5 y 5.6 representan las frecuencias de aparición de cada casilla como casilla inicial o final, respectivamente.

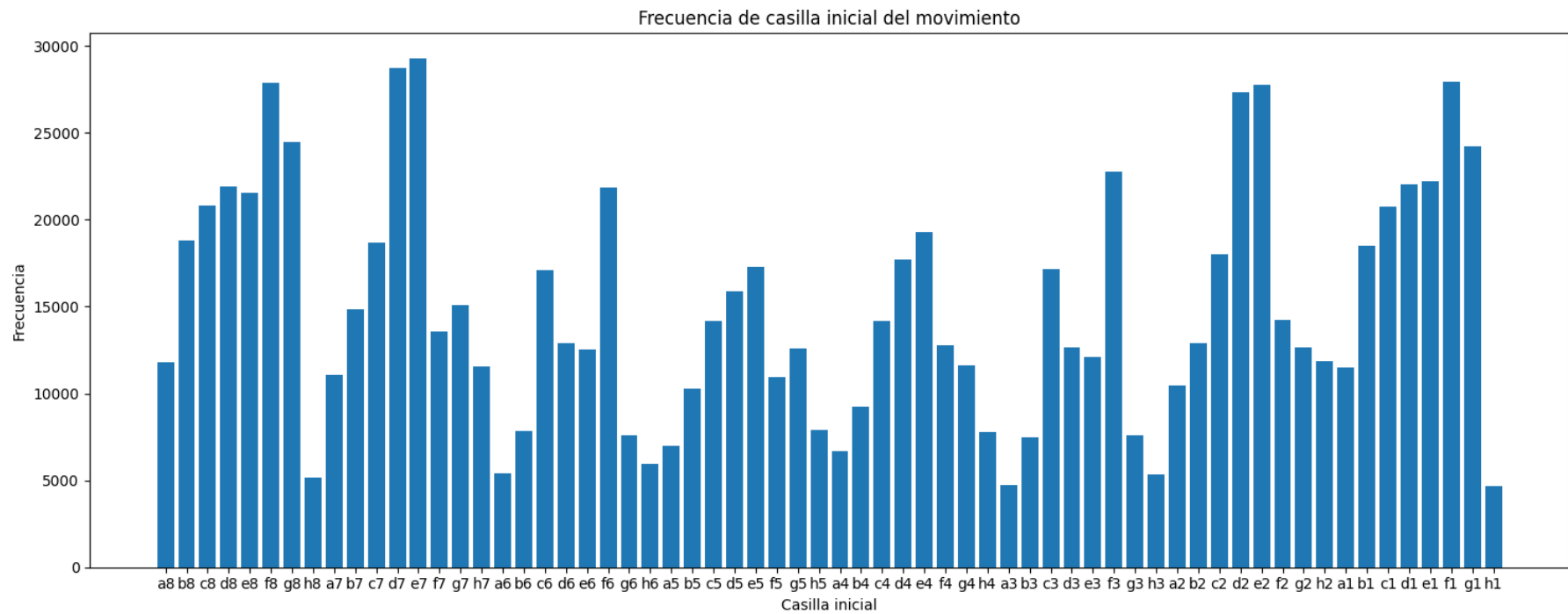


Figura 5.5: Frecuencia de casilla inicial.

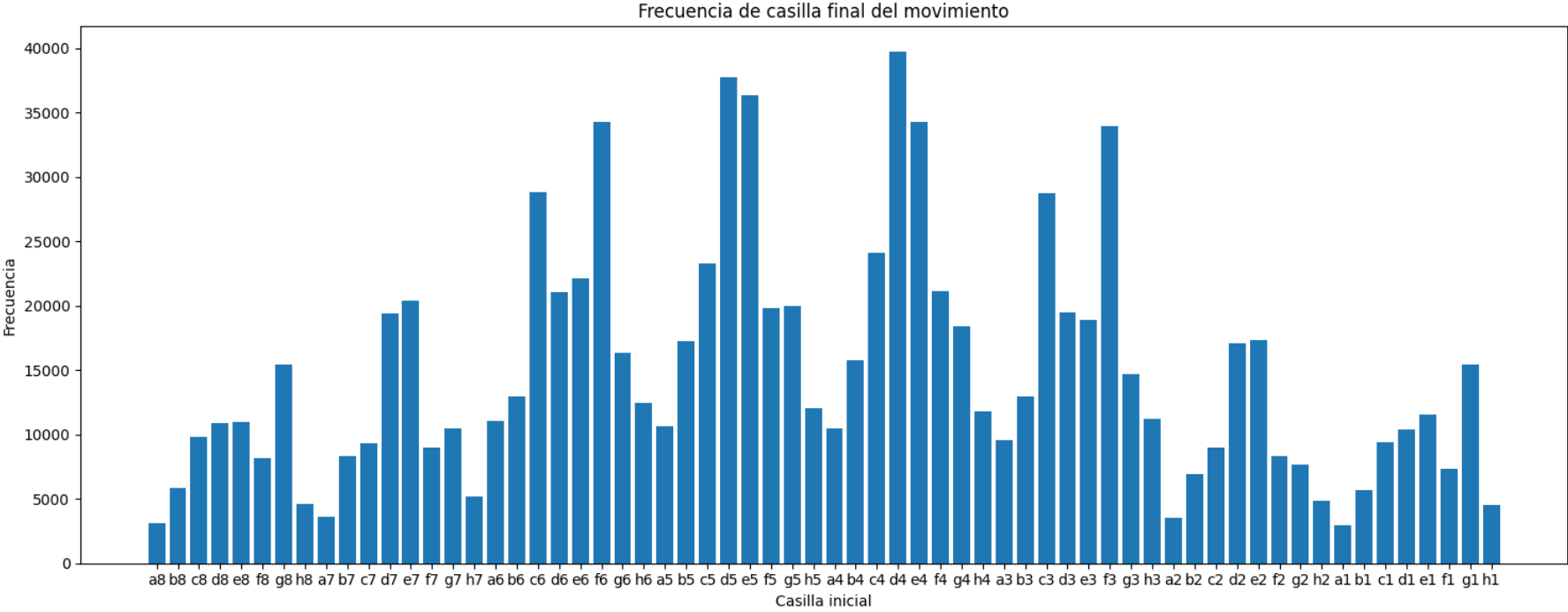


Figura 5.6: Frecuencia de casilla final.

Podemos ver que la distribución de elección de casillas tiende hacia las 2 primeras y últimas filas del tablero, además de cómo se tienden a concentrar los movimientos en las columnas centrales del tablero. También comprobamos que las columnas no están balanceadas, pero aún así son mucho más homogéneas que las del movimiento completo. Por esta aparente simplificación del problema inicial, se decide comenzar prediciendo cada una de estas columnas por separado antes de intentar predecir el movimiento completo. Esto permitirá realizar pruebas con motores más simples, que predigan una de entre 64 casillas, permitiendo una iteración sobre la resolución del problema más fluida.

Finalmente, las características que tendremos disponibles para la resolución del problema durante el desarrollo serán las siguientes, ya descritas anteriormente:

Columna	Tipo
board	Vector(Int)
white_king_rights	Integer
white_queen_rights	Integer
black_king_rights	Integer
black_queen_rights	Integer
ply	Integer
is_check	Integer
move	String
initial_pos	String
final_pos	String
is_blunder_cp	Integer

Tabla 5.2: Tabla “moves” que recoge las entradas del *dataset* transformadas.

5.1.3. Gestión de los datos para el entrenamiento

El *dataset* del que disponemos es dividido en tres subconjuntos, lo cual permite asegurar un correcto proceso de entrenamiento. Así, se toma un 10 % de los datos totales para pruebas. Aunque a priori se podría pensar en reducir este número, debido al gran número de entradas que contiene, se debe tener en cuenta el desbalanceo del *dataset*. Muchas de las jugadas tienen muy pocas representaciones, por lo que se debe intentar que el máximo número de ellas aparezca durante las pruebas. De los datos restantes se reserva otro 10 % para validación, los cuáles condicionarán la toma de decisiones durante la fase de modelado para el ajuste del modelo. El resto queda como datos de entrenamiento, que serán procesados por la red para actualizar sus pesos. Estos tres *datasets* se almacenarán en tres archivos tabulares por separado.

Por último, debemos tener en cuenta una otra consideración. Al haber pasado de una representación de los datos en cadenas de texto a una representación vectorial similar a *One-Hot Encoding*, no se dispone de los recursos *hardware* necesarios para cargar todos los datos a la vez en memoria, por lo que será necesaria una gestión inteligente de los mismos. Para ello, durante el proceso de entrenamiento, se realizará una lectura parcial del *dataset*. Esta lectura

se ha ajustado para que no supere la capacidad total de memoria del equipo del que se dispone. Una vez que se acaben de leer los datos parciales, se proceden a borrar de memoria y cargar los siguientes hasta completar el entrenamiento. Se ha comprobado que este proceso hace que los tiempos de entrenamiento se acorten y sea menos propenso a error.

5.2. Desarrollo de los modelos

A continuación se detallan los modelos implementados durante la elaboración de este trabajo, así como los resultados obtenidos en los entrenamientos de los mismos. Los modelos se exponen en el orden en el que han sido desarrollados, ya que los conocimientos adquiridos y los problemas encontrados han llevado a la mejora iterativa de estos. Todos los modelos han sido implementados mediante la librería *Keras* de *Tensorflow*.

Ya que contamos con una gran cantidad de datos, cada modelo primero se ha probado con una parte pequeña del *dataset*, donde se puede evaluar aproximadamente las repercusiones que distintos cambios y ajustes de hiperparámetros pueden tener en su desempeño. En general, se darán los resultados obtenidos del entrenamiento con el *dataset* completo. Los primeros modelos se entrenarán únicamente para las piezas Blancas, permitiendo agilizar el proceso de modelado. Una vez se tenga la arquitectura final, se entrenará tanto para Blancas como para Negras.

5.2.1. Modelo de predicción de pieza a mover

La primera tarea a resolver en el desarrollo de este proyecto es la predicción de la pieza que va a mover el jugador, lo que quiere decir resolver un problema de clasificación de 64 clases. Resolver este problema más simple nos permitirá asegurar que los modelos que desarrollemos pueden obtener conocimiento a partir de la representación que hemos definido. La arquitectura a alto nivel del modelo va a ser una constante a lo largo del desarrollo. Esta se compone de 2 partes principales:

1. **Una serie de bloque convolucionales formando una CNN** que se encargarán de extraer la información del tablero de juego. Estos bloques aplican *padding* sobre los tensores de entrada, permitiendo que no se reduzca la dimensionalidad espacial del tablero. Si se redujese la dimensionalidad del tablero, siendo este sólo de 8 por 8 casillas, podríamos perder información precisa crucial para la toma de decisiones posterior. En imágenes sí es común reducir estas dimensiones, ya que encontramos resoluciones de imagen muy grandes, donde los elementos que la conforman tienen formas y tamaños muy distintos. Estos bloques convolucionales tendrán distintos métodos de regularización aplicados, como capas de *Batch Normalization* o *Dropout*.
2. **Un *flatten* de las características extraídas.** En este punto se pasa de tener información espacial sobre el tablero a características independientes que servirán de entrada a las siguientes capas.

3. **Conjunto de capas densas formando un MLP.** Estas capas se encargarán de extraer el conocimiento necesario para seleccionar el movimiento deseado a partir de las características extraídas por la CNN. También podrán presentar capas para aplicar la regularización.
4. **Capa densa de salida.** Esta capa tendrá un número de neuronas igual al número de clases del problema que se quiera resolver, normalmente utilizando una función de activación *softmax*. Esta función hace que las salidas de las 64 neuronas sumen 1, permitiendo obtener una distribución de probabilidad de los movimientos a jugar. El movimiento con una mayor probabilidad será el escogido para ser jugado.

Siguiendo este esquema, el primer modelo implementado consta de:

1. 3 bloques convolucionales de 8, 32 y 64 filtros respectivamente, de dimensiones (3, 3), con funciones de activación *relu*. Se aplica *Batch Normalization* y capas *Dropout* de 0,2.
2. 2 capas densas de 256 y 128 neuronas respectivamente, con funciones de activación *relu*. Se añaden capas *Dropout* de 0,2 tras cada capa densa.
3. 1 capa densa de 64 neuronas, el número de casillas en el tablero, utilizando la función *softmax*.

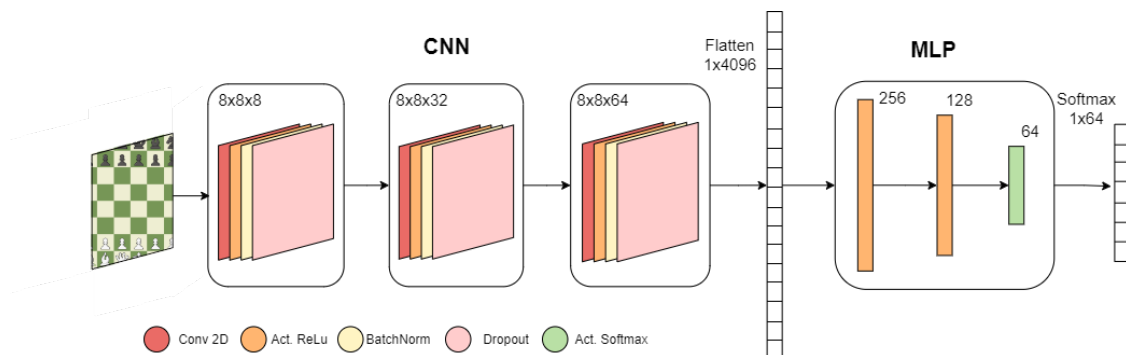


Figura 5.7: Arquitectura de Modelo predicción posición inicial.

Esto nos da un total de aproximadamente un millón de parámetros entrenables, la mayoría dados por la unión de la capa *flatten* y la primera capa densa. Como entrada del modelo, únicamente se espera el tablero de juego.

Al compilar el modelo, se utiliza el optimizador *Adam* y como función de pérdidas la *Categorical Crossentropy*. Esta función de pérdidas espera etiquetas de *One-Hot Encoded*, un vector de longitud igual al número de clases totales que contiene un uno en la posición de la clase a predecir. Por ello, durante el entrenamiento, se aplicará dicha transformación, dándonos más libertad para poder experimentar con otras representaciones en otros modelos, a costa

de algo de tiempo de cómputo durante el entrenamiento. En este momento, se ha probado a utilizar tanto *Categorical Crossentropy* como *Sparse Categorical Crossentropy* con etiquetas numéricas, ambas con resultados similares.

Para cada entrenamiento, se guardan los valores de la función de pérdida y de *accuracy* obtenidos. Estos datos se toman cada cierto número de *steps* realizando una evaluación con un subconjunto de los datos de validación ya que, al contar con tantas entradas, no son necesarias muchas épocas para que el modelo converja, y puede perderse la perspectiva en la tendencia del entrenamiento. Se mantienen las evaluaciones con el *dataset* de validación completo al final de cada época.

El primer entrenamiento con el *dataset* completo de este modelo se realiza con *batches* de tamaño 64, y 10 épocas de entrenamiento en total, dando un total de 61.106 *steps* de entrenamiento. Siempre se utilizará de ahora en adelante la técnica de *Early Stopping* para mitigar el efecto que el *overfitting* pueda tener sobre el modelo, guardando la época con un menor valor para la función de pérdidas. Durante este primer entrenamiento se obtienen los siguientes resultados:

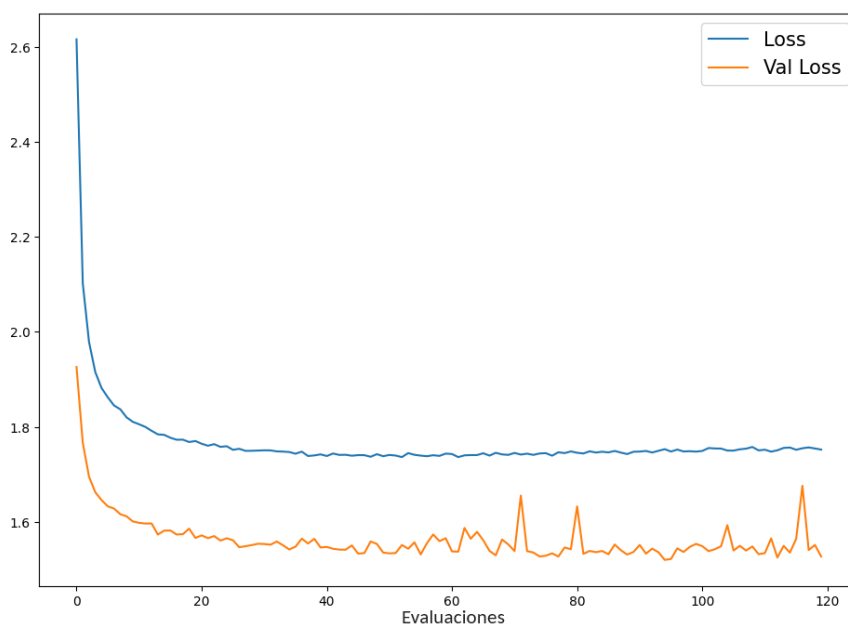


Figura 5.8: Pérdidas de entrenamiento de modelo inicial

Los resultados obtenidos son aceptables. Se consigue una tasa de acierto del 41,5% en la época 1 de entrenamiento, y del 44,25% en la época 8, siendo este el máximo. Para los datos reservados en test se obtiene una *accuracy* del 43,7%, por lo que podemos concluir que nuestros datos de validación son un buen reflejo de los de test.

Como podemos observar en las figuras 5.8 y 5.9, en poco tiempo el modelo no es capaz de progresar con el aprendizaje, quedando la curva de entrenamiento con un rendimiento inferior a la de validación. Esto se debe a que el modelo utiliza capas *Dropout* y a la gran cantidad de datos que debe aprender. Estas capas realizan un “apagado” de un porcentaje de las neuronas

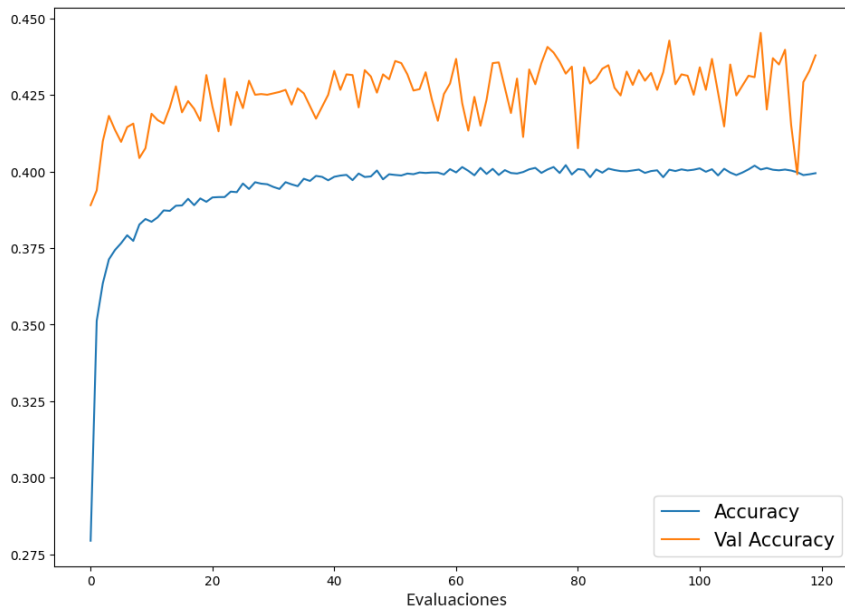


Figura 5.9: Accuracy de entrenamiento de modelo inicial

durante el entrenamiento, haciendo que las restantes no sean tan dependientes unas de otras y el modelo sea capaz de generalizar mejor. Durante la fase de validación, la capa *Dropout* no tiene efectos, por lo que se consiguen mejores resultados que en entrenamiento. Sin embargo, un valor muy alto en estas capas, como el utilizado, lleva al problema que vemos. El modelo no es capaz de aprender correctamente los datos de entrenamiento, por muchas épocas que realice, por lo que apenas llega a sufrir de *overfitting*. Así, la curva de entrenamiento nunca llega a alcanzar a la de validación. Decimos que este modelo sufre de *underfitting*.

Por tanto, nuestra primera mejora consiste en encontrar el valor óptimo de *Dropout*, permitiendo mantener una buena capacidad de generalización en el modelo sin caer en el *underfitting*, lo que será una constante a lo largo del trabajo. Este problema ha derivado de haber utilizado el mismo modelo para un subconjunto de los datos de entrenamiento y para el conjunto completo, ya que con pocos datos sí que se cae en *overfitting*. Si mantenemos exactamente la misma arquitectura pero eliminamos las capas *Dropout* completamente, obtenemos los siguientes resultados que se muestran en las figuras 5.10 y 5.11.

Como se puede observar, la curva ha cambiado. Ahora, la curva de entrenamiento para la función de pérdidas tiene un decrecimiento continuo, sin llegar a un punto sin más aprendizaje. Esto le permite modelar mejor todos los datos de los que dispone, aunque termina empeorando los resultados de validación. Esta curva se considera que presenta *overfitting*, ya que ambas curvas no terminan de converger juntas. Aún así, están muy cercanas entre sí.

Los resultados son mucho mejores que los anteriores. Ya en la primera época de entrenamiento se obtiene una *accuracy* del 47,8% para los datos de validación, llegando hasta un 50,55% en la última época. Recordemos que los datos mostrados en las gráficas solo utilizan un subconjunto de los datos de validación, por lo que presentan más irregularidad. El resultado para test es de un 50,57%, muy cercano al de validación.

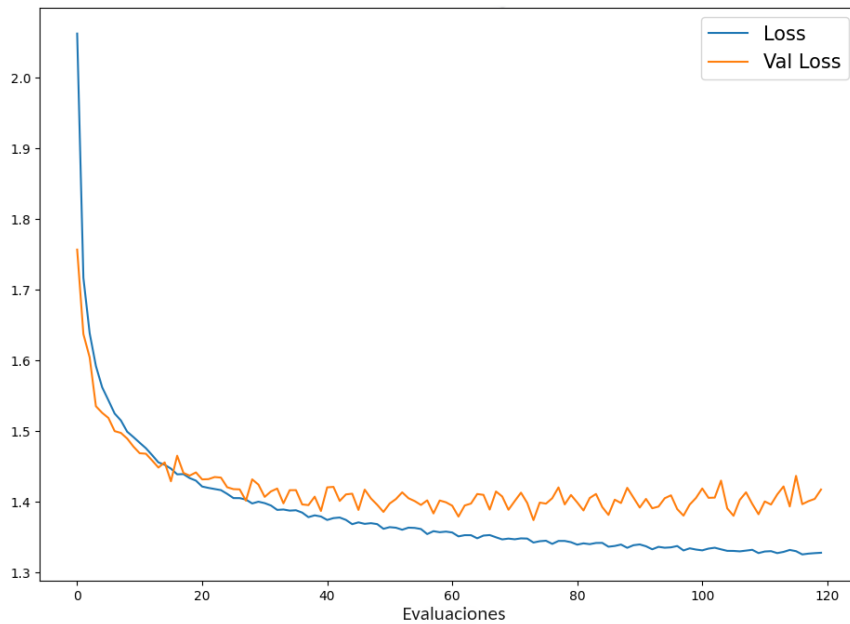


Figura 5.10: Pérdidas de entrenamiento de modelo sin *Dropout*

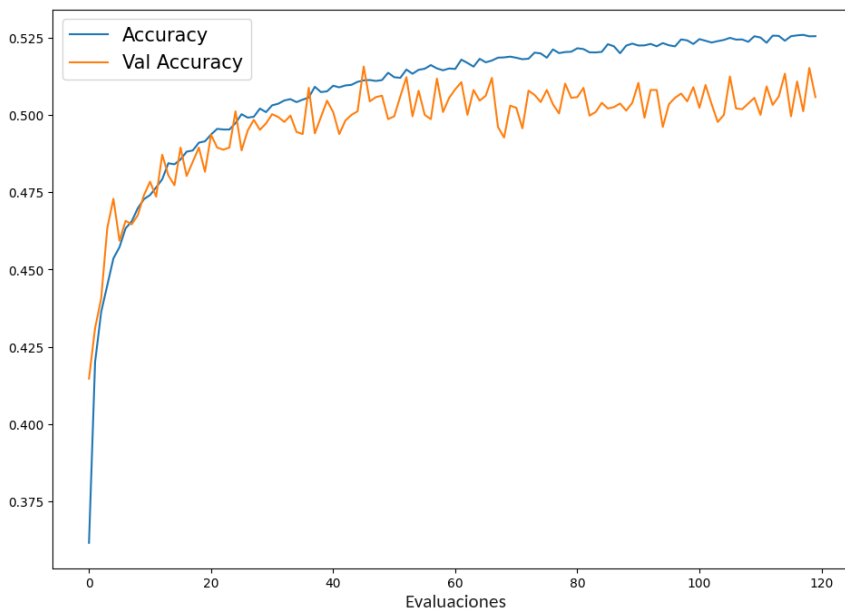


Figura 5.11: Accuracy de entrenamiento de modelo sin *Dropout*

Debido a estos resultados, se toma este modelo como base para la implementación de las siguientes versiones.

5.2.2. Modelos de predicción de posición final

El siguiente paso es ser capaz de predecir la posición a la que se va a mover dicha pieza. Este problema es exactamente igual que el problema anterior, de selección de la pieza a mover, así que se aprovecha para incluir pequeños cambios en la arquitectura de la red y comprobar su efectividad. Estos cambios son:

1. Reemplazo de las funciones de activación *relu* por *sigmoid*. Al probarse esta arquitectura con pocos datos se encuentra unos mejores resultados utilizando estas funciones. Este cambio viene motivado por el uso de valores negativos en la entrada del modelo, comprobando así si estas funciones son mejores tratando con ellos. Sin embargo, no se ve que estas mejoras se trasladen al modelo entrenado con los datos completos, por lo que se eliminarán más adelante.
2. Uso nuevamente de capas *Dropout* tanto en los bloques convolucionales como en el MLP, pero con un valor de 0,1. Se busca encontrar el punto exacto de regularización.

Además, a parte de estos pequeños cambios, se añade una modificación sustancial al modelo. Esta segunda versión del modelo consta de dos salidas, una para la posición de la pieza a mover y otra para la posición a la que moverla. Estas son dos predicciones por separado de 64 clases cada una, pero ya se empieza a acercar a la predicción del movimiento completo. La Figura 5.12 muestra la arquitectura de la red de manera esquemática.

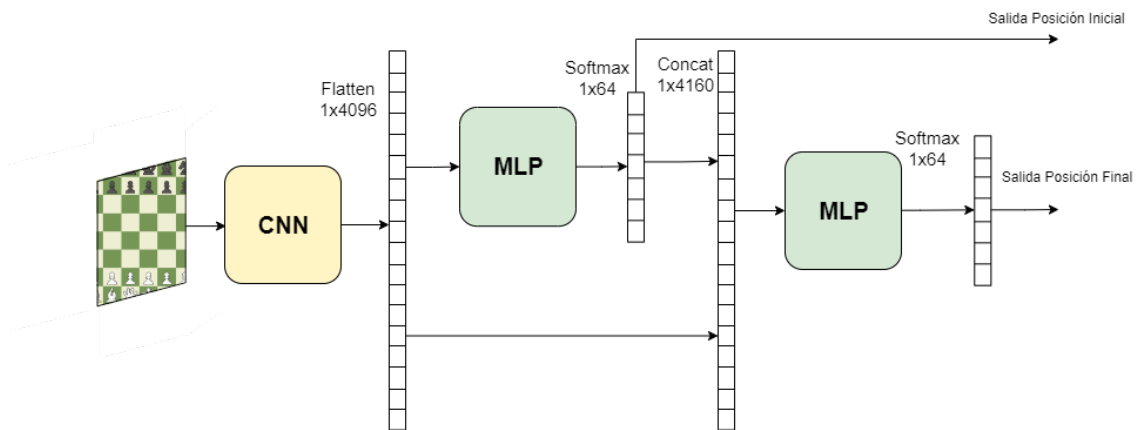


Figura 5.12: Arquitectura de Modelo predicción de posiciones inicial y final.

Para realizar la predicción de la posición final, se añade un paso de concatenación entre el *flatten* de las características extraídas por la CNN y las 64 salidas de la capa densa de la predicción del movimiento inicial. Esta concatenación, con un total de 4160 características, sirven como entrada a un MLP igual al de la predicción inicial, con otra capa densa de 64 neuronas con función de activación *softmax*. Esta arquitectura asume la dependencia entre la pieza a seleccionar y la posición a la que moverla, condicionando la segunda a la primera, lo cual tiene una serie de ventajas y desventajas. Por un lado, es cierto que puede ser información útil, pero una predicción errónea de la pieza inicial podría perjudicar a la segunda parte de la red.

El entrenamiento se realiza con los mismos hiperparámetros que el anterior, pero aumentando el número de épocas a 20 y monitorizando una función de pérdidas para cada salida. Los resultados obtenidos son los siguientes:

Tras la primera época de entrenamiento se consigue una *accuracy* del 46,15 % para la posición inicial y un 35,73 % para la posición final. Tras 16 épocas se consiguen los mejores

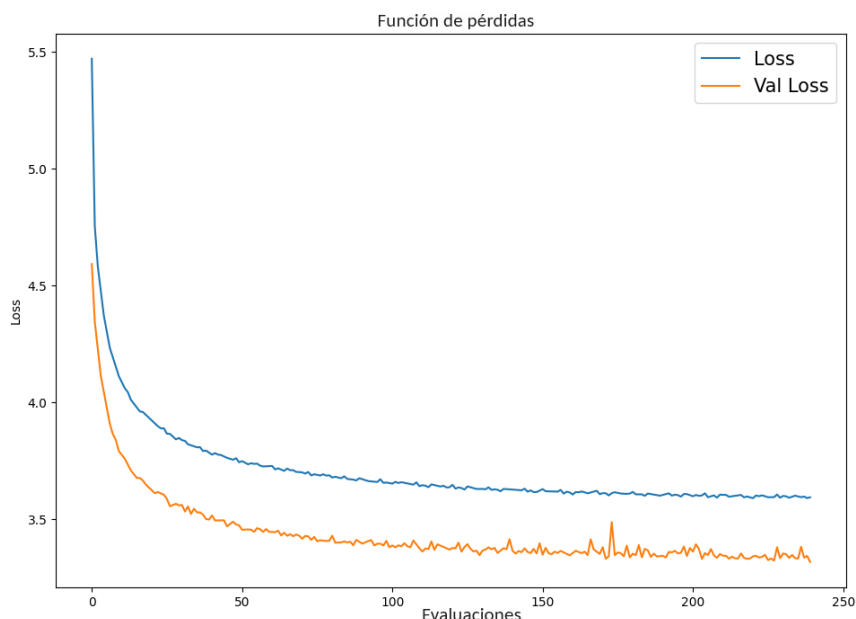


Figura 5.13: Proceso de entrenamiento de predicciones inicial y final

resultados del entrenamiento, un 50,79 % para la posición inicial y un 40,88 % para la posición final.

En la gráfica de la Figura 5.13 podemos ver que se vuelve a presentar un *underfitting*, debido a las capas de *Dropout* como ya se sabe. Por tanto se deben seguir ajustando en futuras iteraciones. Pese a ello, los resultados son muy similares a los obtenidos por el modelo inicial, por lo que podemos concluir que utilizar ambas funciones de pérdidas para ajustar los pesos del modelo permite mejorar el modelo en su conjunto, no solamente las capas respectivas a su salida correspondiente. Esto será muy importante en futuras iteraciones.

Aún así, no podemos decir aún que estemos prediciendo el movimiento completo. En el mejor de los casos, en el que siempre que se acierte la posición final se haya acertado la posición inicial, lo cual es poco probable que ocurra, el modelo llegaría a un 40,88 % de acierto. Si asumimos una distribución homogénea de ambas tasas de acierto, lo cuál seguramente tampoco se cumpla, se obtendrá un resultado del $50,79 \% \cdot 40,88 \% = 20,76 \%$. Podemos concluir que la capacidad actual de nuestro modelo se encontraría en este margen. Es necesario seguir mejorando la arquitectura de la red para obtener unos resultados aceptables.

Antes de continuar con la predicción del movimiento completo se realizó una evaluación que permitiese conocer algunos datos de relevancia sobre el rendimiento del modelo. En la Figura 5.14 se muestra en azul el número de veces que el modelo ha predicho cada una de las casillas como posición a la que mover la pieza, y en naranja el número total de veces que aparece esa casilla en el *dataset*.

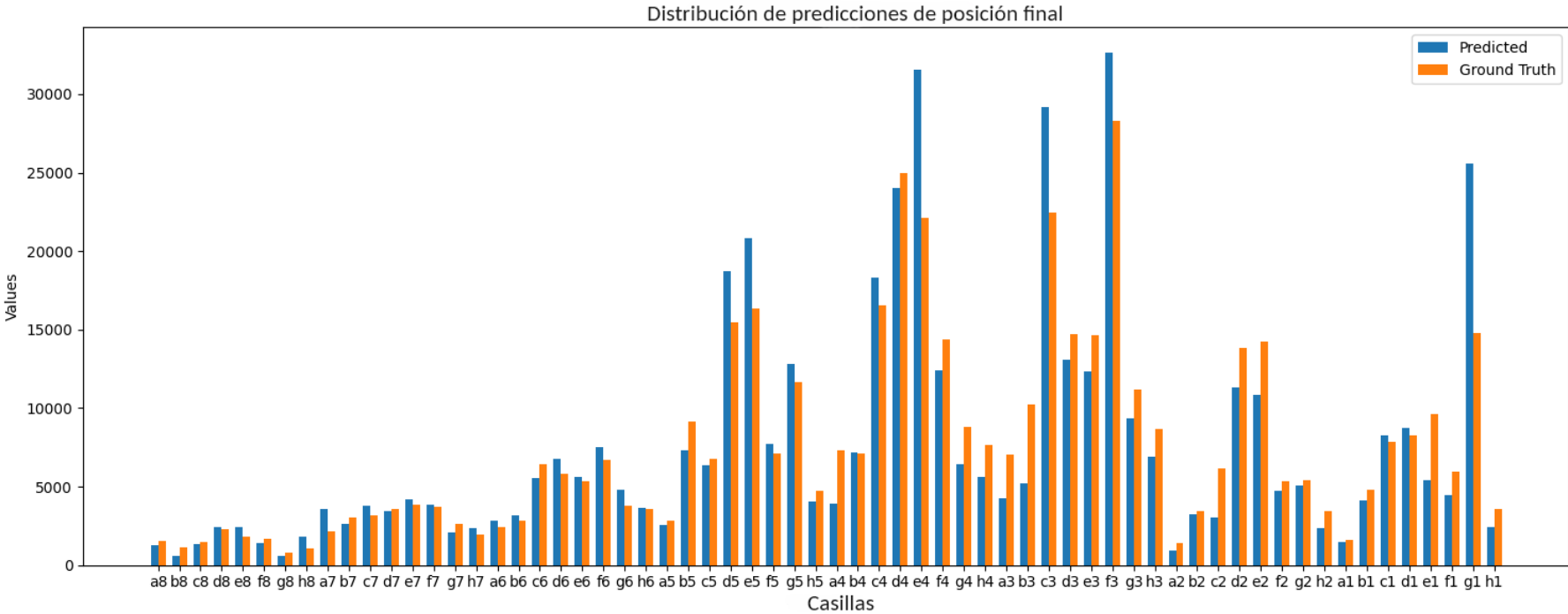


Figura 5.14: Distribución de predicciones de posición final utilizando el segundo modelo.

Se puede comprobar que, aunque no es igual, sí que ambas distribuciones tienen una forma similar, tendiendo las posiciones predichas a tener unos picos mayores. Esto se debe a que son posiciones que aparecen en las primeras jugadas de una partida, donde es más difícil predecir qué hará el jugador. Estos primeros movimientos se denominan la *apertura*, y es imposible saber de antemano qué apertura jugarán las Blancas. Así se explican picos como el de la casilla *e4*, ya que esta es la posición final de la apertura más común en ajedrez. Por otro lado, se puede ver que el modelo no se limita a utilizar únicamente las casillas más populares, sino que es capaz de intentar predecir todas ellas.

Otro dato relevante a la hora de programar un motor de ajedrez es su capacidad de sugerir jugadas que sean válidas según las reglas del ajedrez. Tenemos que tener en cuenta que no se está limitando de ninguna manera la salida del modelo, por lo que podría escoger cualquier combinación de dos casillas como salida. En este respecto se obtiene que:

- El 98,9 % de las casillas que se propone mover contienen una pieza blanca con algún movimiento legal disponible.
- El 97,3 % de las veces se predice como posición final una casilla a la que se puede mover alguna pieza blanca legalmente.
- Si concatenamos las dos salidas de la red, de manera que obtengamos un movimiento completo, el movimiento predicho es legal el 74,9 % de las veces.

Estos resultados demuestran que la red es capaz de inferir las reglas del juego a partir de los patrones que encuentra en el *dataset*. Con estos datos, se realiza una primera evaluación de la capacidad de la red para predecir movimientos completos, siguiendo el acercamiento mencionado de concatenar ambas salidas. En este frente, se obtiene una *accuracy* del 31,45 %, la cual se enmarca en el margen mencionado previamente. Para los movimiento marcados como posiciones en jaque, este porcentaje sube hasta un 39,9 %.

5.2.3. Modelos de predicción de movimiento completo

La siguiente etapa del trabajo consiste en extender el modelo ideado en la fase anterior para aumentar su capacidad de predecir movimientos completos. Para ello, se iteró sobre un subconjunto de las jugadas de Blancas, permitiendo terminar los entrenamientos en pocos minutos. La única diferencia que tienen estos modelos con los entrenados sobre los datos completos es que tienen un mayor *Dropout*, compensando así el *overfitting*.

En un principio, se comenzó intentando aprovechar la posibilidad de concatenar ambas salidas del movimiento anterior. En vez de representar los movimientos como una de entre 4096 clases, se pensó en representarlos como 2 vectores de longitud 64 concatenados, pudiendo así simplificar la tarea. Este primer modelo es exactamente igual al utilizado en el apartado anterior, pero con una única salida de tamaño 128. Para el entrenamiento se mantienen todos los hiperparámetros especificados previamente, únicamente diseñando una función propia de *accuracy* que se adapte a la nueva salida del modelo.

El siguiente paso consistió en entender este problema como un problema de clasificación multi-etiqueta. Estos problemas se caracterizan por la posibilidad de predecir más de una clase para una misma entrada. En nuestro caso, podemos ver el problema como una clasificación donde siempre se quieren predecir dos clases distintas para cada entrada.

La principal diferencia que conlleva esta perspectiva es la función de activación utilizada en las capas de salida, así como la función de pérdidas utilizada. La función *softmax* propicia que una de las salidas sea muy superior al resto, por lo que ambas clases correctas a predecir “competirían” por ser elegidas. Por ello, las funciones *softmax* se cambian por capas sigmoides, y la función *Categorical Crossentropy* por la *Binary Crossentropy*. Esto permite interpretar cada salida individual de la red como una probabilidad entre cero y uno para esa clase. Siguiendo este acercamiento, se definen 2 modelos distintos:

1. El modelo de predicción de movimientos base, pero únicamente cambiando la función de activación de las últimas capa densas *softmax* por sigmoides.
2. Un modelo nuevo donde las salidas de los MLPs para las posiciones inicial y final con funciones sigmoides se concatenan con las características extraídas por la CNN para servir de entrada a un último MLP con 2 capas densas, de 512 y 128 neuronas, también con funciones de activación sigmoide. Este modelo ya consta de una única salida.

Como última propuesta de este desarrollo inicial, se define un modelo con una única salida de 4096 clases, volviendo por tanto al uso de la función de activación *softmax* y a la *Categorical Crossentropy*. Para ello, se selecciona la última arquitectura descrita anteriormente, donde los MLPs de las posiciones inicial y final no se retroalimentan y se concatenan para un MLP final del movimiento.

Los resultados obtenidos por todos estos modelos se recogen en la siguiente tabla. Para realizar esta comparación, se han escalado el número de neuronas de cada uno para que tengan un número similar de parámetros entrenables:

Modelo	Salida	Crossentropy	Accuracy	Movimiento legales	Mejor época
Modelo 1	Concatenada	Categorical	15,58 %	53,35 %	13
Modelo 2	Concatenada	Binary	16,68 %	56,95 %	14
Modelo 3	Única(128)	Binary	16,89 %	55,85 %	14
Modelo 4	Única(4096)	Categorical	17,1 %	58,3 %	10

Tabla 5.3: Comparación de arquitecturas entrenadas con un subconjunto de los datos

Ante estos resultados iniciales, se puede ver que el Modelo 4 es el más prometedor de los desarrollados. Algo que llama mucho la atención de ese modelo es que empieza a sufrir *overfitting* antes, en torno a la época 10, en la cual ya obtiene los mejores resultados. Esto puede llevar a pensar que este modelo al entrenarse con el *dataset* completo puede responder mejor, ya que este efecto se contrarrestará. Por tanto, esta es la arquitectura de partida elegida para las siguientes iteraciones.

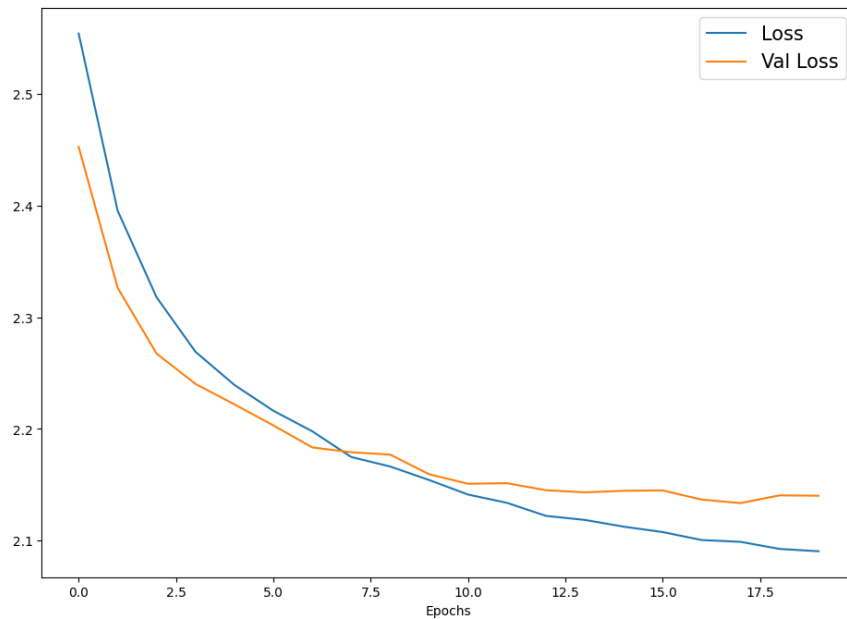


Figura 5.15: Pérdidas de entrenamiento de modelo 4

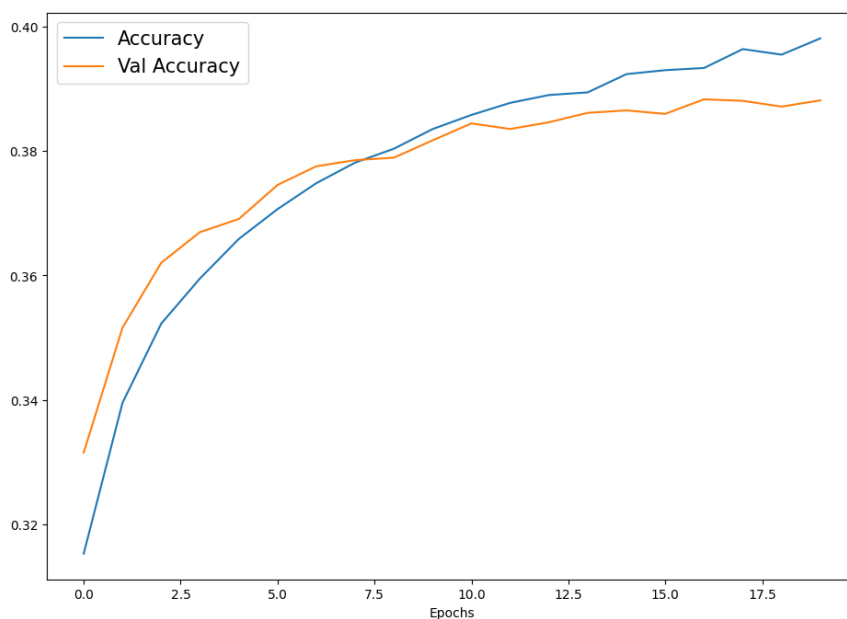


Figura 5.16: Accuracy de entrenamiento de modelo 4

Las figuras 5.15 y 5.16 muestra los resultados del entrenamiento para el Modelo 4. Se comienza el entrenamiento con un 33,1 % de *accuracy* en la primera época para los datos de validación. Recuperando su mejor época, la número 18, se obtiene unos resultados del 38,78 %, una mejora con respecto al 31 % obtenido por el modelo que concatena ambas salidas. También podemos ver que el modelo sigue sufriendo algo de *overfitting* utilizando *Dropout* de 0,1, por lo que todavía debe estudiarse como reducirlo.

Durante este proceso de desarrollo, van surgiendo algunas ideas para mejorar el modelo, las cuales se detallan a continuación:

- Se reemplazan todas las funciones de activación de las capas ocultas que anteriormente eran sigmoides por *LeakyReLU*, al comprobarse por ensayo que proporcionan un mejor rendimiento. Estas funciones son capaces de lidiar con valores negativos sin tantos problemas como las *ReLU* habituales.
- Se propone mejorar la CNN de la red. Para empezar, cada bloque convolucional contendrá 2 capas convolucionales junto con una *Batch Normalization* y su función de activación. Además, se añaden capas de *Squeeze and Scite* junto con *skip connections* antes de la última función de activación. Estos dos cambios mejoran mucho la capacidad de la CNN para extraer información valiosa del tablero.
- Se escala el tamaño de la red, añadiendo 4 de los bloques convolucionales previos mencionados y doblando el número de neuronas de las redes MLP.

Tras aplicar estos cambios, se realiza un entrenamiento, del cual se muestra su progreso en las figuras 5.17 y 5.18. El modelo consigue un *accuracy* del 41,2 %, mejorando a la versión anterior, aunque se vuelve a sufrir de *underfitting*, como se puede apreciar en la figura.

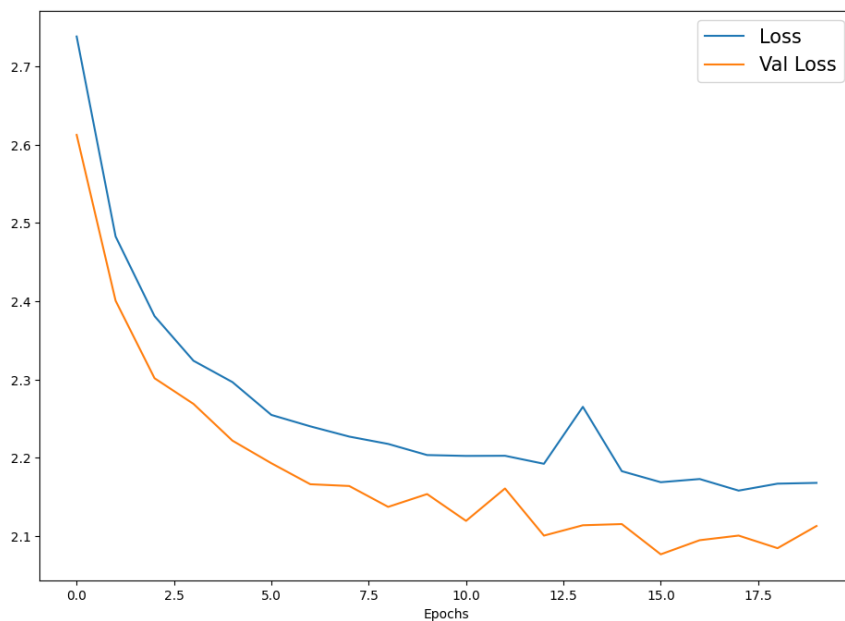


Figura 5.17: Pérdidas de entrenamiento de modelo 4 mejorado

Como siguiente paso, se decide recuperar una idea que se tuvo durante la evaluación del primer modelo capaz de predecir la posición final. Como se mencionó antes, utilizando una monitorización de las funciones de pérdidas para las salidas de los MLPs de las posiciones inicial y final, se mejora el conjunto de la red, y no sólo a cada salida por separado. La idea intuitiva de esto es ayudar a la red a ordenar la información que fluye por ella, permitiendo su mejor comprensión por las capas posteriores. Además, también serviría para ayudar a que la red converja en un menor número de épocas, ya que puede resolver las distintas predicciones de distinta dificultad y retroalimentarse. La Figura 5.19 muestra de manera esquemática la red desarrollada.

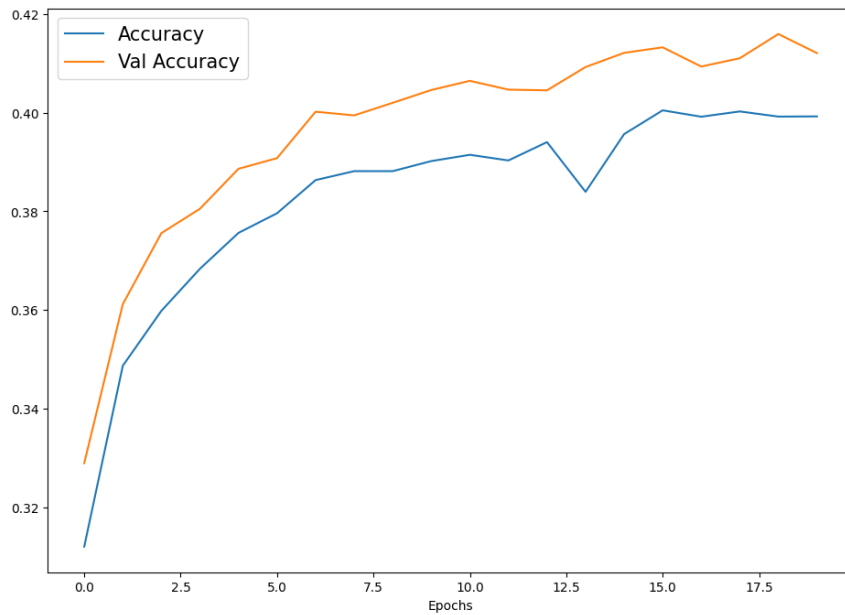


Figura 5.18: Accuracy de entrenamiento de modelo 4 mejorado

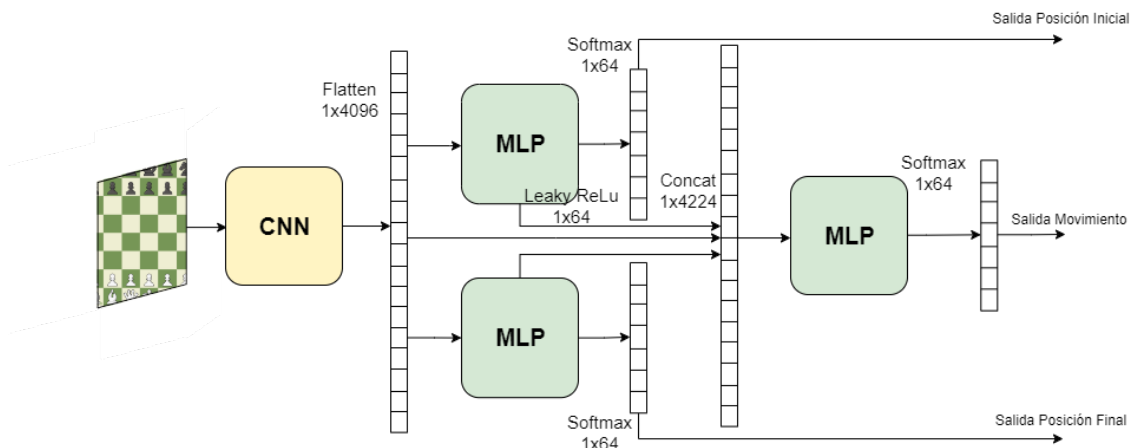


Figura 5.19: Arquitectura de Modelo predicción de movimiento completo.

Durante el primer entrenamiento de la red, se elimina el *Dropout* para lidiar con el *underfitting* y se obtiene que la mejor época de entrenamiento corresponde a la época 11, mucho más rápido que en los modelos anteriores. A continuación se decide ajustar el *Dropout* dejándolo únicamente en los bloques convolucionales, ya que a partir de la época 12 la red pierde capacidad de generalización. Así, el modelo obtiene una tasa de acierto del 45,36 % para la predicción del movimiento, además de un 57,26 % y un 48,77 % para las posiciones inicial y final respectivamente. Se obtienen así los mejores resultados de entre todos los modelos desarrollados, por lo que será el utilizado para la generalización del modelo para Blancas y Negras. Su proceso de entrenamiento se representa en la Figura 5.20.

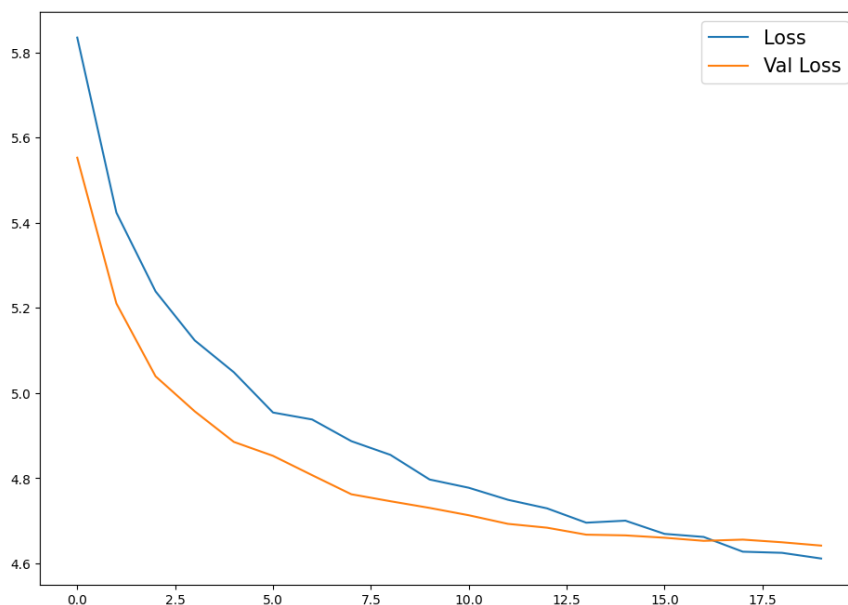


Figura 5.20: Proceso de entrenamiento de modelo final.

5.2.4. Modelo de predicción de error

Para predecir si el jugador cometerá un error en la posición dada, se ha usado un arquitectura similar a la descrita en los apartados anteriores para la predicción de posiciones y movimientos. La arquitectura consta de una CNN de 3 bloques convolucionales y un MLP de 3 capas densas. El modelo termina en una capa densa con una única neurona y función de activación sigmoide. Así, se asegura de devolver un valor entre 0 y 1, que represente la probabilidad de que el jugador falle al hacer el movimiento.

Para el entrenamiento se han utilizado los mismos hiperparámetros, además de la función de pérdidas *Binary Crossentropy*. Debido al gran desbalanceo del *dataset*, recordemos que aproximadamente el 11 % de las entradas están clasificadas como fallos, es muy probable que el modelo aprenda únicamente a dar como “negativas” el 100 % de las entradas, asegurándose un 89 % de acierto. Para compensar esto, se ha aplicado una ponderación de clases, haciendo así que un fallo en la predicción de la clase minoritaria tenga un efecto mayor en el cálculo de la función de pérdidas que en la clase mayoritaria.

Como podemos comprobar en la Figura 5.21 y 5.22, el proceso de entrenamiento es muy errático, sin apenas obtener conocimiento a partir de la primera época. En esta primera época, el modelo consigue un 63,6 % de *accuracy* sobre los datos de validación, pero no mejora de manera constante a lo largo de las épocas. Sobre el *dataset* de test, el modelo obtiene un 65,4 %. Sin embargo, la tasa de aciertos no es un valor fiable a la hora de representar la calidad de un modelo para la predicción de datos tan desbalanceados. En el apartado de evaluación se detalla el proceso seguido para estudiar en profundidad el desempeño del modelo.

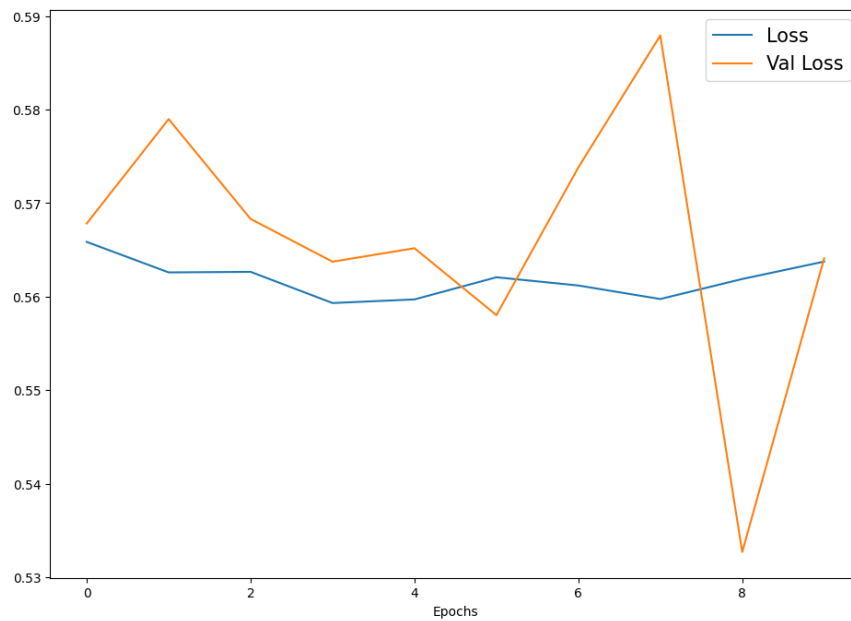


Figura 5.21: Pérdidas de entrenamiento de modelo de predicción de error

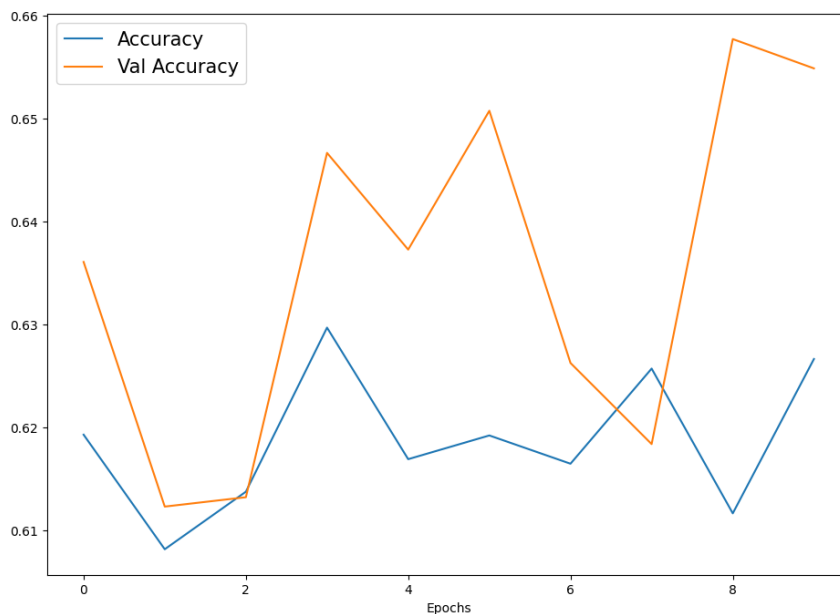


Figura 5.22: Accuracy de entrenamiento de modelo de predicción de error

5.3. Evaluación y comparación de resultados

A continuación se presentan las distintas evaluaciones y métricas aplicadas sobre el modelo desarrollado finalmente, tanto de predicción de movimientos como predicción del error. Además, se realizará la comparación con los motores principales del estado del arte a la hora resolver la tarea de predicción de movimientos humanos.

5.3.1. Evaluación del modelo de predicción de movimientos

A continuación se describen los resultados obtenidos por el modelo descrito finalmente en el apartado 5.2.3, entrenado sobre el conjunto completo de los datos, tanto de Blancas como de Negras. Este modelo ha sido entrenado durante 20 épocas con aproximadamente 7,7 millones de jugadas, y se está evaluando con aproximadamente 950 mil. Esto permite que la mayoría de las jugadas posibles en el ajedrez sean evaluadas. Ya que algunas de estas sólo tenían una entrada en nuestro *dataset*, pueden no aparecer en los datos de prueba, quedando un total de 1792 de los 1914 movimientos. Estos movimientos que faltan no deberían tener una gran repercusión en la evaluación debido a la poca representación que tienen.

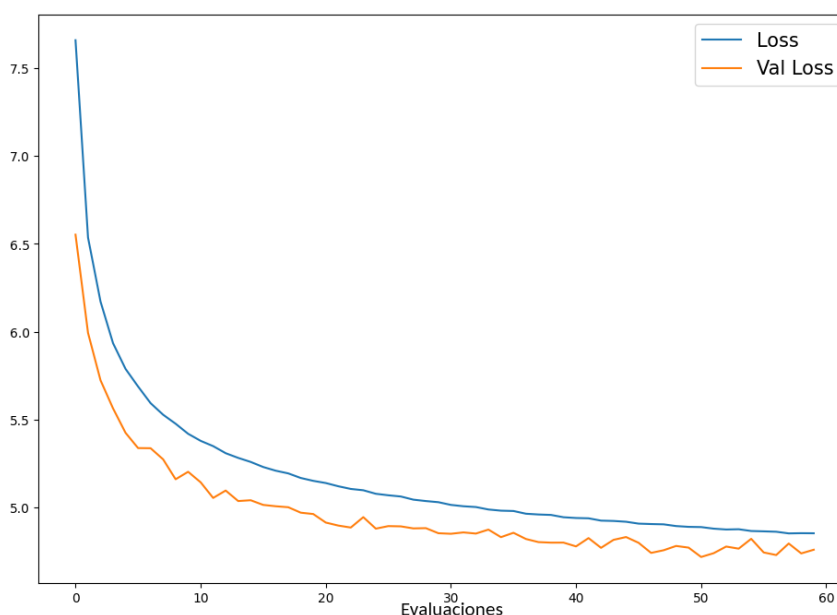


Figura 5.23: Proceso de entrenamiento de para Blancas y Negras.

Para empezar, el modelo obtiene una tasa de acierto del 44.42 % para la predicción del movimiento, menos de un 1 % menos que al entrenarse únicamente para Blancas. En cuanto a las otras dos salidas, se consigue predecir la pieza a mover correctamente el 56,1 % de las veces, y la posición final un 47,49 %. Estos resultados son muy buenos, teniendo en cuenta el gran número de clases posibles y la naturaleza no determinista de las etiquetas que queremos predecir ante una misma entrada. Sin embargo, ya que el *dataset* no está balanceado al haber movimientos más populares que otros, necesitamos utilizar otras medidas que nos permitan estudiar mejor la distribución de las predicciones.

Si observamos cada clase por separado, se podría considerar cada una de ellas como un problema de clasificación binaria. En estos problemas, se puede definir cada resultado como “verdadero positivo” (TP), “falso positivo” (FP), “verdadero negativo” (TN) y “falso negativo” (FN), atendiendo a la relación entre la predicción y el resultado esperado. A partir de estos 4 términos se definen multitud de métricas que nos permiten evaluar el modelo desde otras perspectivas. Las que utilizaremos a continuación son la Precisión, la Sensibilidad (Recall) y la

F1 Score, que no es más que la media armónica de las dos anteriores:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1 Score} = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Visualizar estas métricas para cada clase es muy útil, ya que muestran si el modelo tiene tendencia a únicamente predecir correctamente las clases más representadas. Los resultados obtenidos se resumen en la siguiente tabla:

Métrica	Precisión	Sensibilidad	F1 Score
Media ponderada	44.29 %	44.41 %	43,43 %
Media no ponderada	40.93 %	38.27 %	38.35
Desviación típica	0,11	0,11	0,11
P.25	34,73 %	34,78 %	31,91 %
P.50 (Mediana)	41,03 %	41,06 %	38,94 %
P.75	47,29 %	47,32 %	45,46 %

Tabla 5.4: Evaluación del modelo por clases.

Analizando estos resultados, podemos ver que los valores para la Precisión y la Sensibilidad son muy cercanos entre sí, aunque no muy altos. Esto nos dice que, por lo general, el modelo no está sesgado hacia únicamente predecir correctamente una serie de clases, obviando las otras. Sí que es cierto que se tiene un mejor rendimiento para las clases mayoritarias, ya que podemos comprobar que la media ponderada de las métricas es mayor que la no ponderada. Aún así, en nuestro contexto de juego de ajedrez este no es un problema crítico. Otros problemas, como los diagnósticos médicos sí que requieren una gran capacidad de distinguir las clases minoritarias. Ese problema sí que es más parecido al caso de predicción de errores, el cual veremos posteriormente.

Otras métricas importantes que hemos presentado anteriormente son, por ejemplo la tasa de movimientos válidos que sugiere el modelo. A continuación se comparan los resultados del primer modelo desarrollado con el actual:

Modelo	Pieza legal	Casilla destino legal	Movimiento legal
Concatenación de casillas (B)	98,9 %	97,31 %	74,89 %
Predicción del movimiento (B)	99,69 %	99,45 %	99,23 %
Predicción del movimiento (B&N)	99,52 %	99,22 %	98,9 %

Tabla 5.5: Evaluación de movimientos legales

Se puede ver así que el entrenamiento del modelo sobre los movimientos finales ha permitido que el modelo conozca mejor las reglas del juego. Además, se puede ver que el uso de más

funciones de pérdida consigue no sólo obtener buenos movimientos legales, sino aumentar las capacidades del modelo para predecir la pieza y casilla destino individualmente mucho mejor.

Por último, se quiere comparar el *accuracy* del modelo con el de Maia y otros modelos, ya que son los referentes en el campo. Según la Figura 4 de [McIlroy-Young et al. \(2020\)](#), estos son los resultados del estado del arte para este problema, seleccionando los que tiene un mejor rendimiento para 1700 puntos de ELO:

	Propio	Maia 1700	Stockfish Depth 15	Leela 3200
Accuracy	44,41 %	52,5 %	40 %	45,1 %

Tabla 5.6: Accuracy de los modelos del estado del arte.

Podemos comprobar de los resultados de la Tabla 5.4 que el rendimiento de nuestro modelo es bueno en comparación con el resto de modelos estudiados. Para leer correctamente estos datos se deben hacer varias consideraciones:

1. Stockfish y Leela no son motores *human-like*. Esto quiere decir que su propósito es encontrar el mejor movimiento posible. Sin embargo, tienen versiones reducidas que a menudo se utilizan como herramientas de aprendizaje para novatos. Estos modelos, deliberadamente omiten series de jugadas en sus cálculos o ponderan las posiciones de manera errónea, intentando imitar el comportamiento del humano. Conforme sube el nivel de los jugadores, vemos que la *accuracy* de estos modelos aumenta, ya que los jugadores suelen jugar con más frecuencia el mejor movimiento.
2. Maia 1700 sí es un motor *human-like*, pero ha sido entrenado con un total de 410 millones de jugadas, ya que disponen de más recursos de con los que se cuenta para la elaboración de este trabajo. Aún así, nuestro modelo entrenado con apenas 9 millones y medio de jugadas, es capaz de aproximarse al rendimiento de Maia, pudiendo considerarse un buen resultado. Además, de estos resultados podemos concluir que la inclusión de perceptrones multicapa para la extracción de conocimientos a partir de la red CNN da muy buenos resultados, ya que es nuestro principal cambio con respecto a Maia, además del uso de varias funciones de pérdidas durante el entrenamiento. Estas redes permiten aprovechar mucho mejor los datos utilizados, pocos en comparación con el resto de modelos, a costa de tener un mayor peso, ya que necesita de muchos parámetros entrenables que almacenar.
3. Los datos de test para nuestro modelo no son los mismos que los utilizados con el resto de modelos. Según se detalla en el artículo se utilizan 200.000 partidas para realizar el test, lo que pueden suponer un total de 8 millones de entradas, mientras que nosotros contamos con menos de 1 millón. Aún así, según se ha comprobado, la distribución de movimientos debería ser muy similar, añadiendo más diversidad de jugadas anómalas.

5.3.2. Evaluación del modelo de predicción de error

Finalmente, se realiza la evaluación del modelo de predicción de errores. Debido al desbalanceo del *dataset* ya mencionado previamente, no se esperan unos resultados excepcionales para este motor. Sin embargo, esta evaluación puede permitir aún el ajuste del modelo para mejorar los resultados.

El modelo tiene una *accuracy* del 65,42 %, algo baja para únicamente contar con 2 clases. Sin embargo, las métricas de Precisión, Recall y F1 Score pueden darnos una información más exhaustiva. Estas tres métricas se pueden calcular a partir de la denominada Matriz de Confusión que se presenta a continuación:

	Predicho negativo	Predicho positivo
Esperado negativo	276.521 (64 %)	155.306 (36 %)
Esperado positivo	11.638 (23 %)	39.348 (77 %)

Tabla 5.7: Matriz de confusión. Se expresa el porcentaje con respecto al total de instancias esperadas de esa clase.

Como podemos ver, el modelo clasifica con mayor probabilidad ambas clases con su respectiva etiqueta. Sin embargo, es muy común que muchas de las entradas que no contienen error sean clasificadas erróneamente (el 36 %). Para balancear este comportamiento a continuación se introduce el concepto de las curvas ROC y PR.

El concepto básico de estas curvas es calcular cómo varían distintas métricas de rendimiento en función de un umbral para el que se considera que una entrada es positiva, en este caso si es un error. La curva PR debe su nombre a las métricas que pone en relación, la Precisión y la Sensibilidad. De manera intuitiva, si se selecciona un umbral muy bajo, el modelo será muy sensible, por lo que habrá muchos falsos y verdaderos positivos. Por el contrario, si se aumenta el umbral, el modelo será más preciso, disminuyendo el número de falsos positivos y verdaderos positivos. La curva ROC, por otro lado, relaciona la Especificidad con el Ratio, o Razón de Falsos Positivos. La Especificidad puede definirse como $1 - \text{Sensibilidad}$. Estas curvas tienden a ser diagonales para clasificadores aleatorios, y a tener un área de valor 1 si son clasificadores perfectos. Esto quiere decir que existe un valor umbral posible a partir del cual se pueden separar perfectamente los casos positivos de los negativos. Es por ello que el área bajo la curva ROC es tan popular como medida de evaluación en problemas de clasificación binaria (AUC ROC).

Los resultados tras la selección de umbrales son los siguientes:

Método	Umbral	Accuracy	Precision	Recall	F1 Score
Umbral por defecto	0,5	65,42 %	0,2	0,77	0,320
Curva ROC	0,534	68,33 %	0,21	0,73	0,327
Curva PR	0,617	74,55 %	0,23	0,6	0,334

Tabla 5.8: Comparación de distintos umbrales

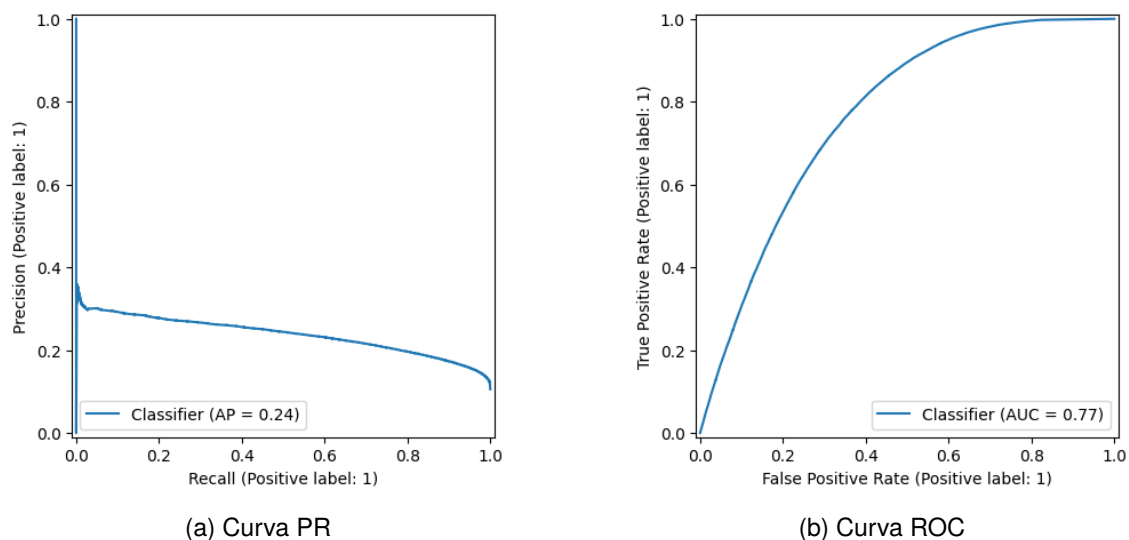


Figura 5.24: Curvas PR y ROC para la predicción de error

La Tabla 5.8 muestra cómo, al ir aumentando el umbral, la Precisión aumenta en detrimento de la Sensibilidad. Esto también implica un aumento considerable de la *accuracy*, ya que se clasifica más fácilmente la clase mayoritaria como negativa. La elección entre cualquiera de estos posibles umbrales es totalmente dependiente del problema que se quiera resolver. En el contexto en el que nos encontramos, creemos que el umbral más conveniente es el que maximiza la F1 Score, proporcionado por la curva PR. La elección de este se debe a darle más importancia a la Precisión del modelo que a la Sensibilidad. No clasificar tableros de error cuando se debería no es tan grave como que un tablero sea clasificado como de error y no sea verdad. Aún así, es un tema debatible y con mucha profundidad y posibles razonamientos válidos.

5.4. Despliegue del modelo

Para permitir el uso del modelo al público y poder utilizarlo para analizar partidas de ajedrez, se pretende su publicación como *bot* en la plataforma web Lichess. Para ello, se debe implementar una API que, ante un tablero expresado en formato FEN, la red devuelva un movimiento en formato algebraico extendido. Antes de realizar este despliegue como *bot* de Lichess, para realizar toda la implementación de la gestión de las entradas y salidas del modelo, se ha integrado con la librería de ajedrez *python-chess*. Esta librería permite generar tableros de ajedrez con los que interactuar, con funciones auxiliares que facilitan tareas como mover un pieza, permitiendo así jugar en local contra el motor para ser probado antes de abrirlo al público.

5.4.1. Integración con Python-Chess

Para realizar esta integración se han seguido los siguientes pasos. Casi todo este desarrollo será común con la integración del motor en Lichess, y por tanto aprovechado una vez

comprobado su correcto funcionamiento.

1. **Inicializar el tablero de juego.** Esto colocará todas las piezas en sus posiciones iniciales a la espera de un movimiento por parte de Blancas.
2. **Adaptar el tablero de entrada.** Ya que la librería *python-chess* nos devuelve la distribución del tablero en formato FEN, podemos seguir el proceso aplicado durante el procesamiento inicial de los datos para obtener su representación como vector multidimensional. Este vector sirve de entrada al modelo.
3. **Adaptar la salida de la red.** El modelo devolverá un vector unidimensional con la distribución de probabilidades de la jugada a realizar. A esta salida se le debe aplicar una transformación para forzar que la salida sea una jugada válida. Para ello, se listan todos los movimientos legales en la posición y se escoge la salida con mayor probabilidades dentro de las permitidas por las reglas.
4. **Traducción a notación algebraica.** Con esta selección realizada se traduce la salida a formato algebraico extendido y se mueve la pieza. Tras esto se espera una entrada por consola de comandos del jugador.
5. **Lectura del movimiento del jugador.** El jugador introduce la jugada deseada en la consola y se mueve la pieza. Así, se vuelve otra vez al punto 2 hasta que la partida concluya.

En la figura 5.25 se muestran los primeros 5 movimientos de una partida.

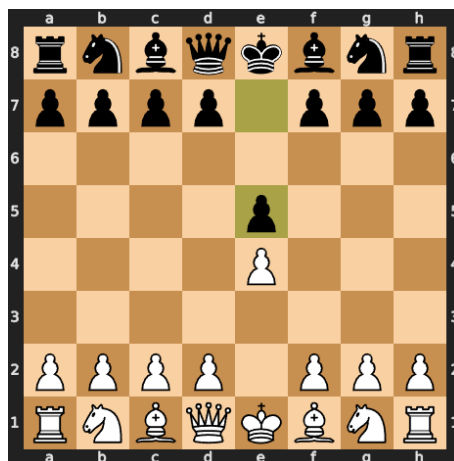
5.4.2. Despliegue en Lichess

Para realizar la integración del motor en Lichess, se ha utilizado el repositorio oficial *lichess-bot*, el cual implementa gran parte de la funcionalidad necesaria para realizar esta conexión. Principalmente, este repositorio permite abrir un canal de comunicación cada vez que se acepte jugar una partida con otro jugador, por el cual llegará la posición actual del tablero, además de otros datos como el tiempo de reloj. También se implementan otras funcionalidades a priori más accesorias, como el poder escribir por el chat de la partida o proponerle al otro jugador tablas.

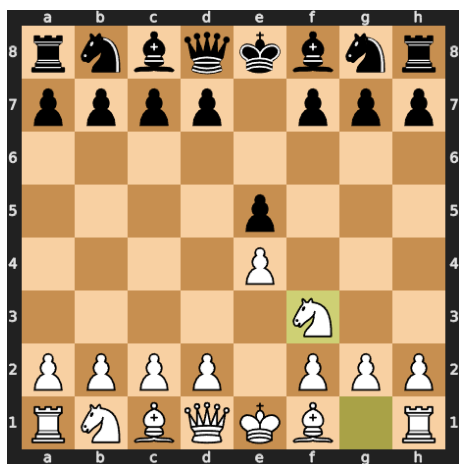
Para integrar un nuevo motor se debe extender la clase *ExampleEngine* que viene por defecto, e implementar la función *search*. Esta función es la que recibe como parámetros la información del canal de comunicación. La clase que se ha implementado se encarga de inicializar el modelo y responder con un movimiento ante la posición actual del tablero. Ya que esta clase se instancia una vez por cada partida que se comience, se ha implementado un patrón *Singleton* simple que permite cargar el modelo una única vez, permitiendo así su uso para múltiples partidas al mismo tiempo. El comportamiento seguido por el motor de ajedrez es muy similar al descrito en el apartado anterior.



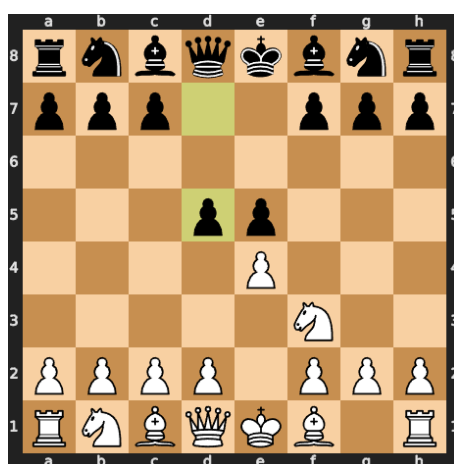
(a) Modelo juega e4. Apertura más común.



(b) Jugador juega e5. Respuesta típica a e4.



(c) Motor juega Kf3. Amenaza al peón de e5.



(d) Jugador juega d5. Error evidente, ya que ambos peones están sin protección.



(e) Motor juega exd5. La mejor jugada. Podría haber comido con caballo, pero no aprovecharía el error.

Figura 5.25: Ejemplo de primeros 5 movimientos en una partida.

Una vez se ha integrado la lógica del motor de ajedrez en la clase *custom* implementada, se procede a crear una cuenta de *bot* en Lichess. Una vez hecho esto, al inicializar la aplicación, esta se conectará a la cuenta que designemos con el API Token que hayamos obtenido al crear la cuenta y podrá recibir peticiones de partidas. Además, a través de la web de Lichess, podemos entrar en la cuenta de *bot* y desafiar a otros motores que estén activos en ese momento.

Para probar el motor, se ha enfrentado contra los *bots* de Maia que hay publicados en Lichess, que corresponden a los modelos Maia 1100, Maia 1500 y Maia 1900. La puntuación de ELO que tienen estos motores en la modalidad de juego *Rapid* son de 1433, 1533 y 1679 puntos respectivamente, habiendo jugado centenares de miles de partidas. Por lo tanto, podemos ver que aunque el modelo imite las jugadas de jugadores de 1900 puntos de ELO, no consigue llegar a ese nivel, pero sí que es mejor que los modelos de 1100 y 1500 puntos. Aún así esta diferencia a la alza o a la baja puede deberse a multitud de factores. En modalidades de juego más cortas, como *Bullet*, tienen aproximadamente 100 puntos más cada uno, debido a que son modelos capaces de jugar más rápido que un humano.

Al probar nuestro modelo las primeras veces, vemos que no es capaz de ganar apenas ninguna partida, ni siquiera contra Maia 1100, por lo que necesitamos hacer un análisis en profundidad para entender el problema. Analizando varias partidas contra estos motores, sacamos las siguientes conclusiones:

- **El motor comienza prácticamente siempre ganando las partidas.** En los primeros 10 o 15 movimientos, nuestro motor es capaz de obtener una ventaja posicional con respecto a su contrincante. Incluso, contra las versiones de Maia 1100 y 1500, es capaz de sacar un peón de ventaja en la apertura.
- **El motor hace jugadas de calidad muy dispar en el medio juego.** A partir del movimiento 15 aproximadamente, el motor es más errático y hace jugadas que son claramente malas, lo que se denomina *blunder* en inglés, las cuáles no deberían darse con tanta frecuencia en jugadores de 1700 puntos. Esto se debe a que el medio juego es cuando más posiciones distintas pueden alcanzarse, ya que aún hay muchas piezas en el tablero que han sido movidas de sus posiciones iniciales. A partir del punto en el que el motor hace una muy mala jugada, es casi imposible que pueda ganar o empatar la partida.
- **El motor no es capaz de hacer mate a partir del movimiento 20.** Los patrones de mate son complejos, pero fáciles de estudiar para un jugador de 1700 puntos. Debido a que estas posiciones no son tan frecuentes en nuestro *dataset* como las aperturas, es fácil que el modelo no sepa llevar a término estas jugadas, además de que presentan muchas posibilidades igual de buenas, por lo que le es complicado decidir por una. También se ha comprobado que Maia a veces presenta este problema.
- **El motor no es capaz de predecir la repetición de una posición** en la partida, lo cual, por las reglas del ajedrez, acaba en tablas. El motor de ninguna manera va guardando

el progreso de la partida a lo largo del tiempo, por lo que posiciones de ventaja clara acaban en tablas por este motivo.

- **El motor realiza la jugada de enroque sin problemas.** Por tanto, no hace falta a priori su integración como parámetro de entrada del modelo, permitiendo así no complicar los datos.
- **El motor es muy rápido en la elección del movimiento.** Al no necesitar de ningún algoritmo de búsqueda, el modelo es capaz de realizar las jugadas más rápido que Maia funcionando en el sistema descrito anteriormente. Sin embargo, para partidas muy rápidas, cuando a Maia le queda menos de un segundo de reloj, cambia su comportamiento y realiza movimientos de manera casi instantánea. El motor que se ha implementado para este trabajo sólo tiene un modo de funcionamiento, por lo que suele perder por tiempo ante otro *bots* en estas modalidades.
- **El motor actualmente es totalmente determinista.** Esto quiere decir que ante una posición siempre realizará la misma jugada. Por lo que se comprobó, Maia también es bastante determinista una vez se sale de la apertura, por lo que a veces se han llegado a repetir partidas enteras.

Ante esta información, se decide realizar una mejora al motor sin modificar el modelo implementado. Los tres problemas más importantes a resolver son la inconsistencia del motor, la incapacidad de hacer mate en finales de juego y el no poder predecir la repetición de posiciones. Para atajar estos problemas se decide implementar un algoritmo que permita al motor reevaluar el movimiento elegido. Para ello se hará uso del motor Stockfish, que permite evaluar posiciones de juego de manera clásica, buscando la mejor jugada posible. El comportamiento será el siguiente:

1. Al principio de la partida se inicializa un tablero mediante la librería *python-chess*, el cual permite llevar todo el histórico de la partida.
2. Se procesa la posición actual del tablero tal y como se ha venido haciendo hasta ahora, que representa la distribución de probabilidad para todos los movimientos posibles. A los movimientos que sean ilegales se les asigna una probabilidad de 0. Luego, se ordenan de mayor a menor probabilidad.
3. Se selecciona el primer movimiento. Se simula la realización de este movimiento mediante una copia del tablero inicializado en el paso 1 y se pide a Stockfish que evalúe la posición. Esta evaluación se denota mediante centipeones. Una desventaja de un peón equivale a -100 centipeones, una posición de mate forzado se ha evaluado como +-10.000, y una posición de tablas como 0 centipeones. Esta evaluación se resta con la evaluación de la posición actual real del tablero, para así calcular cómo de bueno o malo ha sido el movimiento.

4. Se designa un umbral, para el cual un movimiento se considera *blunder*, en nuestro caso un movimiento inadmisibles para un jugador de 1700 puntos, y se compara la evaluación obtenida con el umbral.
5. Si el movimiento es mejor que el umbral determinado, se selecciona este movimiento y se procede de manera habitual, además de realizar la jugada en nuestro tablero inicializado en paso 1. Si el movimiento es peor, ese movimiento se descarta y se vuelve al paso 3, pero utilizando el siguiente movimiento en la lista ordenada de probabilidades obtenida en el paso 2. Este paso se repite un número definido de veces, para que no se evalúen todos los movimientos legales del tablero.
6. Si ninguno de los movimientos que ha propuesto el modelo superan el umbral, se consideran igualmente malos, y se realiza el movimiento con mayor probabilidad, sin tener en cuenta la evaluación.

Para nuestro caso hemos definido como *blunder* una pérdida de 300 centipeones, lo que equivale a perder una pieza en un sólo movimiento sin obtener nada a cambio. Para la selección del resto de parámetros que condicionan este comportamiento, se realizan evaluaciones sobre los datos de test. Se comprueba que, para una evaluación máxima de 5 movimientos distintos y un tiempo máximo de evaluación para Stockfish de 0,05 segundos, se incrementa la tasa de aciertos del modelo de 44,41 % a 46,33 %, por lo que se puede concluir que el uso de Stockfish de manera auxiliar no desvirtúa al modelo para imitar a jugadores de 1700 puntos. Aún así, es importante tener en cuenta que este procedimiento hace al modelo más robusto ante errores, por lo que puede complicar la predicción de errores graves comunes en jugadores de este nivel.

Tras estos cambios, el desempeño del motor contra las distintas versiones de Maia cambia totalmente. En partidas de unas 40 jugadas, el motor hace uso de Stockfish una media de 3 veces, principalmente a partir del movimiento 20. Principalmente, se hace uso de Stockfish en posiciones de victoria claras del final de juego, donde se busca el mate. Durante el medio juego, Stockfish es capaz de evitar movimientos erráticos ocasionales, la mayor parte de las veces eligiendo el segundo o tercer movimiento propuesto por el modelo. Además, al utilizar un tablero inicializado al comienzo de la partida para la simulación de los movimientos, el motor es capaz de evitar la repetición de posiciones que lleven a tablas si se lleva ventaja.

Así, se ha conseguido un motor que puede vencer de manera consistente a Maia 1100 y Maia 1500, pero puede empatar o perder contra Maia 1900. Esto nos permite intuir que la evaluación de nuestro modelo puede ser cercana a la que obtiene Maia 1900, recordemos, en torno a 1650 puntos de ELO, lo cual es muy cercano a los 1700 puntos que se buscaban. Aún así, para saber esto con exactitud, el modelo debería jugar cientos de partidas contra jugadores humanos.

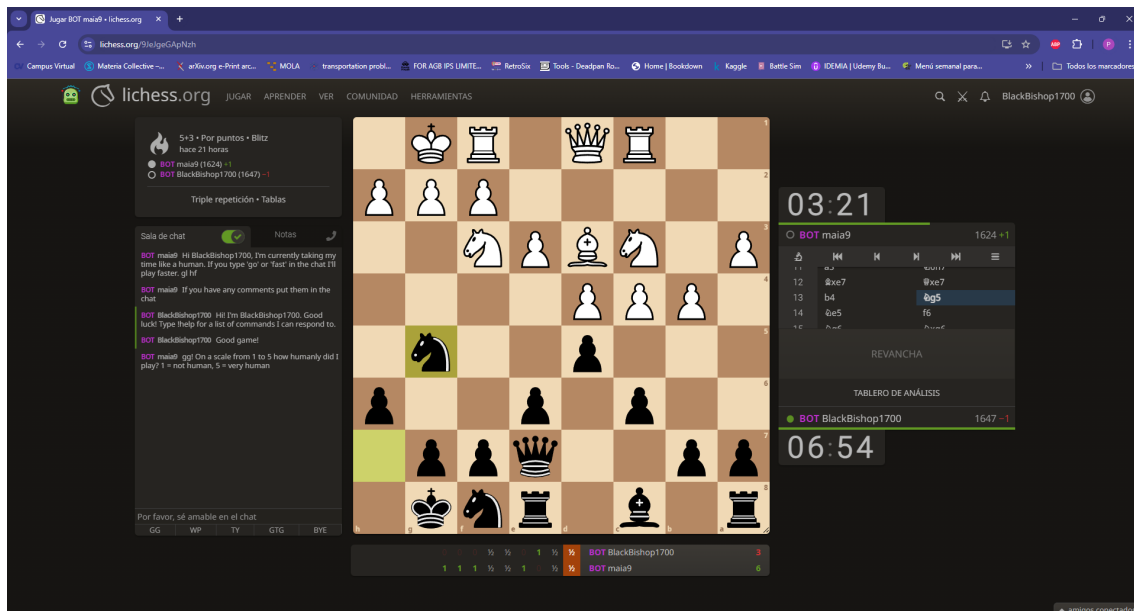


Figura 5.26: Partida entre nuestro motor (Negras) y Maia 1900 (Blancas) terminada en tablas.

Conclusiones, limitaciones y perspectivas de futuro

6

A continuación se presentan las conclusiones extraídas del desarrollo de este TFM. Luego, se presentarán las limitaciones encontradas y posibles líneas de desarrollo futuras que pueden partir de este proyecto.

6.1. Conclusiones

Los principales resultados de este TFM son los siguientes:

- Se han estudiado y comprendido los conceptos teóricos que fundamentan los motores de ajedrez y los motores *human-like*. Esto incluye conceptos de redes neuronales y *Deep Learning*, los distintos procesos de entrenamiento que pueden ser aplicados y los requisitos funcionales necesarios. En concreto, se ha conocido en profundidad la aplicación de entrenamiento supervisado, aplicado en arquitecturas de redes neuronales convolucionales y perceptrón multicapa.
- Se han recopilado datos de jugadores de ajedrez de distintos niveles, y se han procesado para su uso en los modelos desarrollados. Estos datos han sido filtrados para jugadores únicamente de 1700 puntos, además de eliminar datos que se podrían considerar anómalos en el contexto de este problema, como son las partidas excesivamente cortas o largas. Además, se ha procesado la representación del tablero de ajedrez para que sea similar a la representación de una imagen, permitiendo aplicar CNNs para la resolución del problema.
- Se han desarrollado distintos modelos que permiten la resolución del problema de predicción de movimientos y de predicción de error. Se ha descrito todo el proceso de desarrollo de los modelos, justificando las decisiones tomadas que llevan a los resultados obtenidos. Se ha obtenido un modelo que predice para jugadores de 1700 puntos con una tasa de acierto similar a otros motores de ajedrez del estado del arte, utilizando una parte pequeña de los datos de entrenamiento en comparación. Además, el modelo desarrollado es perfectamente aplicable para otros niveles de jugadores, para lo cual sólo es necesario su entrenamiento con partidas del nivel deseado. También se concluye que, tanto la inclusión de perceptrones multicapa en la salida de las CNN, como el uso de múltiples

funciones de pérdidas, mejoran mucho los resultados obtenidos, permitiendo aprovechar mejor los datos con los que se cuenta.

- Se ha adaptado el modelo desarrollado para su uso como *bot* en Lichess. La cuenta de *bot* utilizada se llama BlackBishop1700, y es fácilmente accesible para cualquier otro usuario de la página. Esto permite jugar una partida contra el motor, tanto de Blancas como de Negras. Este motor realiza movimientos mucho más comprensibles para jugadores noveles que otros como Stockfish o Leela, por lo que es muy útil para entrenar si no se dispone de otra persona. Además, las herramientas que ofrece la página permiten repasar los movimientos de la partida y si estos han sido buenos o no. Esto permite que los jugadores aprendan a aprovechar los errores típicos de jugadores de 1700 puntos, ya que también son imitados por el motor. Durante este proceso de despliegue, se han realizado distintos ajustes para obtener un motor con un nivel constante en sus jugadas, utilizando una ligera evaluación de los movimientos propuestos por la red.

La principal mejora que da el motor desarrollado es el permitir a cualquier persona enfrentarse a jugadores del nivel deseado, para ver cómo juegan y las idiosincrasias de cada uno, permitiéndoles personalizar mucho su aprendizaje. Esto brinda grandes ventajas frente al juego multijugador online habitual, ya que este requiere que ambos jugadores sean de un nivel similar, y se realiza siempre desde un carácter competitivo, sin posibilidad de analizar la partida durante el juego sin que se considere trampas. Además, también se ataja el principal problema que supone este tipo de partidas educativas, y es que únicamente el jugador con menos experiencia es el que se beneficia de la misma. Para jugadores noveles, es complicado encontrar jugadores expertos que estén dispuestos a jugar contra ellos, ya que no obtendrían muchos puntos de ELO por vencerles. Al ser el jugador más experimentado un *bot*, este estará siempre disponible para el uso de cualquier jugador primerizo.

6.2. Limitaciones

Para seguir mejorando los resultados del modelo desarrollado, sería necesario incrementar los recursos *hardware* empleados durante el proceso de entrenamiento. La tarea a resolver necesita de grandes cantidades de datos muy diversos, para los cuales su procesamiento y almacenaje necesita de sistemas muy potentes si se quieren terminar en tiempos razonables. Durante las pruebas con distintas arquitecturas, se detectó que el número de bloques convolucionales utilizados aumenta mucho el tiempo de entrenamiento. Por ejemplo, el entrenamiento del modelo final, entrenado tanto para Blancas como para Negras, tardó un total de 7 horas y media, para 10 épocas de entrenamiento y 7,7 millones de entradas. Por tanto, el entrenamiento del modelo para una cantidad de datos equivalente a los utilizados en Maia era inviable.

La mejora de estas condiciones supondría un aumento en la tasa de aciertos del modelo a lo largo de toda la partida. Actualmente es muy competente, sobre todo en los movimientos de apertura, pero poder ser entrenado con más datos del medio juego permitiría contar con un motor más robusto, seguramente acercándose mucho más a los resultados de Maia.

A raíz de esto, ya que los bloques convolucionales aumentan mucho el tiempo de procesamiento, para la arquitectura de las redes se ha ideado intensificar la aplicación de capas densas con muchas conexiones, lo cual ha aumentado mucho el peso de la red, teniendo en torno a 9 millones de parámetros entrenables. Aún así, el tiempo de procesamiento de la red es bastante rápido, permitiendo su uso para partidas en tiempo real, no sólo para la evaluación de posiciones individuales.

Estas mejoras impulsarían además el desempeño del modelo de predicción de errores, ya que requiere de un gran volumen de datos balanceados entre movimientos correctos y erróneos. Este balanceo supone el procesamiento de más datos para su limpieza y recopilación en un único *dataset*. Gracias a los datos publicados por Lichess y preprocesados por el equipo de Maia, esta tarea es viable, pero requiere de muchos recursos. Aún así, se ha realizado todo el proceso de análisis posterior para esta tarea, permitiendo la aplicación de este proceso en el caso de disponer de los recursos necesarios.

6.3. Líneas futuras

Durante el proceso de desarrollo se han ido apuntando distintas líneas de mejora que se pueden seguir en un futuro, desde pequeños cambios o exploración de hiperparámetros hasta cambios en el *bot* de Lichess implementado. Las más importantes o que podrían tener una mayor repercusión en el desempeño de la red son las siguientes:

- Adaptación y entrenamiento del modelo para un conjunto mayor de datos. En comparación con Maia, hemos utilizado unas 50 veces menos entradas. Aún así, se cree que estas posiciones en gran parte serán redundantes, sobre todo durante la apertura, donde el modelo tiene un desempeño actual muy bueno. Se debería intentar aumentar el número de movimientos en el *dataset* que correspondan al medio y final de juego, con el objetivo de intentar no depender de un motor externo para mantener la consistencia de la red.
- Realizar un entrenamiento de los datos balanceados para la predicción de error. Con un buen modelo que realice esta tarea, se podría incluir dentro del motor de Lichess, como complemento o sustituto de Stockfish. Esto puede permitir seguir prediciendo correctamente malos movimientos, mientras se continúa evitando que se cometan fallos garrafales.
- Utilizar representaciones del tablero distintas. Por ejemplo, utilizar la representación de (8,8 12), donde las piezas negras utilizan canales distintos a las blancas. También se podría contemplar el procesamiento de la posición inmediatamente anterior, permitiendo que la red siga una estrategia a lo largo de varios movimientos, ya que cada posición no estaría completamente descontextualizada.
- Ajuste de distintos hiperparámetros durante el entrenamiento. Se deberían probar distintos valores de *Dropout*, regularización L1 y L2, distintos optimizadores e incluso utilizar

un peso de las funciones de pérdida variable. Actualmente, se le da el mismo peso a las 3 salidas de la red en el cálculo de la función general de pérdidas. Sin embargo, sólo se utiliza una de ellas, la que predice el movimiento completo. Así, puede que una vez que el entrenamiento esté llegando a su fin, aumentar la importancia de la salida principal de la red mejore su desempeño, ya que podría estar siendo lastrada por el resto.

- Extensión del *bot* de Lichess para un uso más lectivo explícitamente. El *bot* es capaz de comunicarse con el jugador mediante el *chat* que hay en partida. Este *chat* puede permitir al motor decir si la jugada que ha hecho el otro jugador es buena o mala, o por ejemplo si se encontraba entre los movimientos que él habría jugado, profundizando en la aplicación educativa del mismo.
- Se debería crear una aplicación para la evaluación de posiciones concretas por parte de un usuario. Una vez se tiene la lógica del *bot* de Lichess, su extracción para probar posiciones concretas no debería ser compleja, pero necesita de un tiempo de desarrollo dedicado. Por ejemplo, se podría implementar una aplicación, que, ante una posición en formato FEN, se den los 5 movimientos con más posibilidades según el modelo, con su probabilidad de ser jugado y la evaluación de Stockfish.

Apéndice A



Todo el desarrollo que ha supuesto este proyecto ha sido subido al repositorio de GitHub que se puede encontrar en el enlace <https://github.com/PabloGorSan/TFM>. En él se pueden encontrar todos los resultados obtenidos y las evaluaciones aplicadas. El repositorio se compone de una serie de carpetas que se corresponden con las distintas iteraciones que se han iniciado. Cada carpeta contiene una serie de Jupyter Notebooks que permiten generar los distintos modelos de inteligencia artificial.

Para cada carpeta existen uno o varios Notebooks llamados *analysis*, en los cuales se ha desarrollado toda la evaluación del modelo, conteniendo los cálculos de movimientos legales, los cálculos de *F1-Score* y los análisis de distribución a lo largo de las casillas del tablero. También se incluyen las pruebas para optimizar los hiperparámetros del *bot* de Lichess, variando el tiempo de ejecución de Stockfish y el número de movimientos considerados.

Además del contenido de estas carpetas también se incluye todo el proceso de tratamiento de datos realizado sobre las bases de datos y sobre *dataframes* de *pandas*. Este proceso viene especificado en el apartado 5.1 de esta memoria, y se corresponde con los archivos llamados *full_pipelines.ipynb*. El producto de estos Notebooks son los distintos archivos *.csv*, los cuales son utilizados en el entrenamiento de los modelos. Los datos de los que se parte son fácilmente adquiribles desde la página web de Maia (https://csslab.cs.toronto.edu/datasets/#maia_kdd).

Por último, en el archivo *chess.ipynb* se incluyen las funciones que permiten adaptar el modelo para su uso como motor de ajedrez. En él se hace uso de la librería *python-chess* para inicializar el tablero de ajedrez y realizar los movimientos. Además, también se definen las funciones que permiten utilizar el motor Stockfish para la reevaluación de los movimientos propuestos por la red.

Bibliografía

- Amanatullah (2023). unleashing the power of multi-layer perceptron: Decoding the significance of weights and biases". In <https://medium.com/>.
- Campbell, M., Hoane Jr., A. J., y Hsu, F.-h. (2002). Deep blue. In *Artificial Intelligence* 134.
- Hu, J., Shen, L., Albanie, S., Sun, G., y Wu, E. (2019). "squeeze-and-excitation networks". In *Arxiv*.
- Kumar, S. (2020). convolution neural networks in a nutshell | a-z guide". In <https://www.herevego.com/convolution-neural-nutshell/>.
- Lichess (2024). "distribución mensual de puntos". <https://lichess.org/stat/rating/distribution/rapid> [Accessed: 2024].
- Linscott, G. (2018). Leela zero documentation. <https://lczero.org/dev/> [Accessed: 2024].
- Loaiza, K. (2020). "artificial neural network activation functions". In <https://www.researchgate.net/>.
- McIlroy-Young, R., Sen, S., Kleinberg, J., y Aderson, A. (2020). Aligning superhuman ai with human behavior: Chess as a model system. In *Arxiv*.
- McIlroy-Young, R., Wang, R., Sen, S., Kleinberg, J., y Aderson, A. (2022). Learning models of individual behavior in chess. In *Arxiv*.
- Romstad, T., Costalba, M., Kiiski, J., y Linscott, G. (2022). Stockfish documentation. <https://disservin.github.io/stockfish-docs/> [Accessed: 2024].
- Silver, D., Hubert, T., y Schrittwieser, J. (2017). Mastering chess and shogi by self-play with a general reinforcement learning algorithm. In *Arxiv*.
- Springer, D. (1996). "automated design of both the topology and sizing of analog electrical circuits using genetic programming". In *Artificial Intelligence in Design '96*.
- Toshniwal, S., Wiseman, S., Livescu, K., y Gimpel, K. (2022). Chess as a testbed for language model state tracking. In *AAAI-22*.
- Wirth, R. y Hipp, J. (2000). crisp-dm: Towards a standard process model for data mining". In *PADD'99*.
- Worrall, S. (2018). Los ordenadores nos dicen con quién salir y a quién encarcelar: pero ¿deberían hacerlo? In *National Geographic*.